



SUPPORT FORMATEUR · ÉDITION SEPTEMBRE 2026

Formation Prometheus & Grafana

Guide du formateur — déroulé complet des trois jours, exercices corrigés, démonstrations, anecdotes et points de vigilance

Sommaire

Avant-propos	3
Ce que promet le programme, et où on le fait	3
Les règles que je me fixe	4
Timing indicatif	4
Matériel	4
L'environnement technique	4
Comment lire ce guide	5
Jour 1 — Voir : architecture et collecte	7
9h00 — Accueil et cadrage	7
Module 1 — Pourquoi l'observabilité	8
Module 2 — Architecture de Prometheus et place de Grafana	9
Module 3 — Installation et prise en main	12
Module 4 — Configuration de Prometheus	15
Module 5 — Les exporters	18
TP 1 — Node Exporter : de la machine à Prometheus	19
Module 6 — Instrumenter son application	22
TP 2 — Instrumentation et services tiers	23
17h15 — Récap et quiz du jour 1	26
Jour 2 — Comprendre : PromQL et Grafana	28
9h00 — Rappel et état des lieux	28
Module 7 — PromQL, les fondations	29
Exercices série A — Sélection et agrégation	31
Module 8 — PromQL avancé	32
Exercices série B — Taux, quantiles, jointures	34
TP 3 — Recording rules et tests unitaires	35
Module 9 — Grafana	36
TP 4 — Un tableau de bord paramétrable pour un serveur Linux	40
TP 5 — Le tableau de bord de la boutique, puis dashboards as code	43
Module 10 — Provisioning, utilisateurs, droits, éditions	47
17h20 — Récap et quiz du jour 2	50

Jour 3 — Réagir : alertes, exploitation, passage à l'échelle	51
9h00 — Rappel	51
Module 11 — Philosophie de l'alerting et règles Prometheus	51
Module 12 — Alertmanager	53
TP 6 — Alertes Prometheus et routage Alertmanager	55
Module 13 — Notifications tierces et ChatOps	57
TP 7 — Cas pratique : alerte CPU élevé, notification dans Teams	59
Module 14 — Alerting Grafana	61
TP 8 — Alerting Grafana, de l'interface au code	63
Module 15 — Performances, limites, bonnes pratiques	65
TP 9 — Sauvegarde, restauration, longue durée, sécurité	67
Module 16 — Mise à l'échelle et écosystème	69
17h05 — War game et évaluation finale	70
Annexes	71
Annexe A — Questionnaire d'évaluation (avec corrigé)	71
Annexe B — Aide-mémoire	72
Annexe C — Ressources	73
Annexe D — Dépannage pendant la formation	73
Annexe E — Créer le flux Teams Workflows (pas à pas)	74

Avant-propos

Ce guide est mon support d'animation. Il contient tout ce que je dis, tout ce que je montre et tout ce que les stagiaires font pendant les trois jours, avec les corrigés. Il est volontairement bavard : le jour J, je ne dois jamais avoir à réfléchir à « qu'est-ce qui vient après ? ».

Le fil conducteur des trois jours est une boutique en ligne fictive, `shop-api`, déployée en deux instances et bombardée par un générateur de trafic. Elle est instrumentée pour de vrai (`prometheus_client`), elle a des métriques techniques et des métriques métier, et je peux la casser à la demande (latence, erreurs, CPU, fuite mémoire). Tout ce qu'on apprend s'applique immédiatement à quelque chose de concret qui tourne sous les yeux des stagiaires.

Ce que promet le programme, et où on le fait

Programme officiel	Où	Comment
Présentation, architecture, cas d'usage	J1 matin	Modules 1 et 2
Rappels d'installation, prise en main de l'environnement	J1 matin	Module 3 + exercices 1.1 à 1.4
Configuration (fichiers, service discovery)	J1 matin	Module 4 + exercices 1.5 à 1.8
Exporters, intégration de services tiers	J1 après-midi	Module 5 + TP 1 (Node Exporter) + TP 2
Cas pratique Node Exporter	J1 après-midi	TP 1
PromQL de base	J2 matin	Module 7 + série A
PromQL avancé, recording rules, optimisation	J2 matin	Module 8 + série B + TP 3
Grafana : source de données, panels, variables, transformations, annotations, liens	J2 après-midi	Module 9 + TP 4 + TP 5
Provisioning, dashboards as code	J2 après-midi	Module 10 + TP 5 (fin)
Utilisateurs, droits, orgs, teams, dossiers ; OSS vs Enterprise	J2 fin	Module 10 + exercices 2.30 à 2.33
Cas pratique dashboard serveur Linux paramétrable	J2 après-midi	TP 4
Alertes Prometheus, Alertmanager	J3 matin	Modules 11 et 12 + TP 6
Intégrations tierces (Alertmanager, PagerDuty, Slack)	J3 matin	Module 13
Teams via Workflows, GitHub	J3 matin	Module 13 + TP 7
Cas pratique alerte CPU → Teams	J3 matin	TP 7
Alerting Grafana (unified alerting, contact points, policies)	J3 après-midi	Module 14 + TP 8

Programme officiel	Où	Comment
Performances, limites, bonnes pratiques	J3 après-midi	Module 15
Sauvegarde, restauration	J3 après-midi	TP 9
Mise à l'échelle	J3 fin	Module 16

Les règles que je me fixe

1. **Jamais plus de 20 minutes sans que les stagiaires touchent au clavier.** Chaque module théorique est suivi d'exercices courts. Les gros TP arrivent quand les briques sont posées.
2. **On construit, on ne détruit pas.** Le `prometheus.yml` du vendredi soir est celui du lundi matin enrichi étape par étape. Idem pour Grafana et Alertmanager. À la fin, chacun repart avec une stack complète qu'il a assemblée lui-même.
3. **Le corrigé n'est jamais donné avant d'avoir cherché.** Les stagiaires ont le guide avec les énoncés et quelques indices. Je projette le corrigé après un temps de recherche, jamais avant.

Timing indicatif

Journées de 9h00 à 17h30, pause déjeuner 12h30-14h00, pauses 15 minutes vers 10h45 et 15h45. Soit environ 6h45 de travail effectif par jour. Les durées indiquées dans les modules sont des cibles ; les TP 4, 5 et 6 sont ceux qui débordent le plus souvent, les bonus sont là pour les rapides.

Si le groupe est en retard, ce qui peut sauter sans casser la suite :

- J1 : exercice 1.8 (relabeling), TP 2 partie 5 (bonus cardinalité)
- J2 : série B exercices 2.19 à 2.22, TP 5 partie 4 (heatmap), exercice 2.33 (service account)
- J3 : TP 6 parties 8 et 9 (silences, mute), TP 9 partie 4 (basic auth)

Matériel

- Ce guide (PDF), dossier `docs/formateur/`, et le deck (PowerPoint), dossier `docs/slides/`
- Les trois guides stagiaires, un par jour, dossier `docs/stagiaire/` (PDF ou Markdown pour Notion). Je ne distribue le guide du jour que le matin même.
- Le dépôt GitHub, branche `formation-2026` : <https://github.com/yparent/formation-observabilite-lab>
- Les corrigés (`solutions/`), ce guide et le deck sont sur la branche `formation-2026-formateur`, que je ne distribue pas. La branche `formation-2026` des stagiaires n'en contient aucun.

L'environnement technique

Tout tourne en conteneurs. Plusieurs modes possibles pour les stagiaires, à valider **avant** la formation avec le client :

Mode	Avantages	Points d'attention
GitHub Codespaces	Rien à installer, identique pour tous	Compte GitHub obligatoire, quota gratuit 120 h-cœur/mois (largement suffisant pour 3 jours en machine 2 cœurs, un peu juste en 4 cœurs), les URLs passent par un proxy
Docker Desktop macOS/Linux	Rapide, tout en local	Docker doit être installé et fonctionnel la veille
Docker Desktop Windows	Idem	Backend WSL 2 obligatoire, PowerShell pour <code>lab.ps1</code> , fins de ligne gérées par <code>.gitattributes</code>

Mon conseil : envoyer un mail une semaine avant avec le lien du dépôt et demander à chacun de lancer `./lab.sh up` (ou de créer son Codespace) et de m'envoyer une capture de Grafana. Ça évite de perdre la première heure.

Ce qui doit tourner à 9h00 le jour 1

- Ma propre stack lancée et chaude depuis au moins 30 minutes (les graphiques ont besoin d'historique).
- Le deck projeté, la fenêtre du navigateur avec un onglet par service.
- Un terminal avec le dépôt ouvert.
- Si Teams est utilisé pour le TP 7 : le flux Workflows créé dans un canal de test, l'URL sous la main.

Codespaces : les pièges connus

- Les ports sont redirigés vers des URLs `https://<nom>-<port>.app.github.dev`. Dans Grafana, les liens « localhost » du dashboard d'accueil ne fonctionnent donc pas : il faut passer par l'onglet Ports. Je le dis dès le début.
- La première ouverture d'un port redirigé affiche une page d'avertissement GitHub, c'est normal.
- Un Codespace s'arrête tout seul après 30 minutes d'inactivité. Les données Docker (volumes) sont conservées tant que le Codespace n'est pas supprimé : le lendemain, `./lab.sh up` repart avec l'historique.
- `postStartCommand` relance la stack à chaque démarrage du Codespace.

Docker Desktop : les pièges connus

- Node Exporter voit la machine virtuelle Linux de Docker Desktop, pas le Mac ou le PC. Les chiffres (CPU, RAM, disques) sont ceux de la VM. Je le dis pendant le TP 1, c'est un bon prétexte pour expliquer comment Docker Desktop fonctionne.
- Sur Windows, si `lab.ps1` est bloqué : `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.
- Ports 3000, 5001, 5002, 8080, 9090, 9091, 9093, 9100, 9115, 9121 doivent être libres.
- Sur Mac, `./lab.sh chaos cpu` fait chauffer la VM, pas la machine. C'est suffisant pour l'alerte.

Comment lire ce guide

Chaque module a la même structure :

- **Objectif** et **durée**

- **Ce que je dis** : le fil du discours, avec les analogies et les anecdotes
- **Ce que je montre** : les démonstrations en direct, commande par commande
- **Exercices** ou **TP** : l'énoncé tel qu'il figure dans le guide stagiaire, puis le corrigé détaillé, les erreurs classiques et ce que je vérifie chez les stagiaires
- **Points de vigilance** : ce qui coince en général

Les encadrés « Anecdote » sont mes retours d'expérience. Je les raconte à ma façon, ce ne sont pas des textes à lire.

Jour 1 — Voir : architecture et collecte

Objectif de la journée : à 17h30, chaque stagiaire a une stack qui collecte des métriques système, des métriques applicatives et des métriques de services tiers, il sait lire un fichier `prometheus.yml` sans trembler, et il a écrit ses premières requêtes.

Heure	Séquence	Durée
9h00	Accueil, tour de table, cadrage	30 min
9h30	Module 1 — Pourquoi l'observabilité	35 min
10h05	Module 2 — Architecture de Prometheus et place de Grafana	40 min
10h45	Pause	15 min
11h00	Module 3 — Installation et prise en main + exercices 1.1 à 1.4	50 min
11h50	Module 4 — Configuration + exercices 1.5 à 1.8	40 min
12h30	Déjeuner	
14h00	Module 5 — Les exporters	30 min
14h30	TP 1 — Node Exporter	60 min
15h30	Pause	15 min
15h45	Module 6 — Instrumenter son application	30 min
16h15	TP 2 — Instrumentation et services tiers	60 min
17h15	Récap, quiz, questions	15 min

9h00 — Accueil et cadrage 30 min

Ce que je dis. Je me présente en deux minutes : architecte cloud, quinze ans d'infra, du Kubernetes en banque, en santé, dans l'industrie, et surtout des nuits passées devant des écrans noirs à me demander ce qui se passait. C'est ça qui m'a amené à l'observabilité : pas la technologie, la douleur.

Tour de table, je note sur le paperboard pour chacun : prénom, poste, ce qu'ils surveillent aujourd'hui et avec quoi (Zabbix, Centreon, Datadog, rien...), et **une panne dont ils se souviennent**. Je réutiliserai ces pannes toute la semaine comme exemples. Ça met tout le monde dans le bain et ça me dit tout de suite qui a déjà touché à Prometheus.

Les questions de positionnement (Qualiopi, mais surtout utiles) :

1. Quelle est la différence entre pull et push ?
2. Qu'est-ce qu'une série temporelle ?
3. Sauriez-vous dire si un bug est côté infra ou côté applicatif ?

Ce que je promets pour mercredi soir : « vous saurez diagnostiquer une panne de la boutique en moins de cinq minutes, et vous aurez été prévenus par Teams avant que le client ne râle ».

Logistique : horaires, pauses, le dépôt GitHub, le guide stagiaire du jour (je ne donne que le jour 1), et la règle : on cherche d'abord, je corrige ensuite.

Ce que je montre. Grafana avec le dashboard *TP 5 - Boutique en ligne* du corrigé, tout vert. Puis `./lab.sh chaos errors on`, et je laisse tourner en arrière-plan : à la pause, le dashboard sera rouge. « Voilà ce que vous saurez construire mercredi. »

Module 1 — Pourquoi l'observabilité 35 min

Objectif. Comprendre la différence entre monitoring et observabilité, situer les métriques parmi les autres signaux, et comprendre pourquoi Prometheus a gagné.

Ce que je dis

Les deux pilotes. Le pilote A a un voyant rouge « MOTEUR » qui s'allume. Il sait que c'est grave, il ne sait pas pourquoi. Le pilote B a un écran qui dit « pression d'huile -40 %, vibration sur l'axe Z ». Il coupe le moteur concerné, compense, se pose. Le monitoring classique, c'est le voyant. L'observabilité, c'est l'écran. Notre métier pendant trois jours, c'est de fabriquer l'écran.

Définition utilisable. Un système est observable quand on peut comprendre son état interne à partir de ce qu'il expose vers l'extérieur, sans avoir à le rouvrir. Concrètement : quand la question « pourquoi c'est lent ? » a une réponse en moins de cinq minutes.

Les trois signaux.

- Les *métriques* : des nombres horodatés. Peu coûteuses, agrégeables, parfaites pour les tendances et les alertes. C'est notre sujet.
- Les *logs* : des événements textuels. Riches, mais chers à stocker et à chercher.
- Les *traces* : le parcours d'une requête à travers les services. Indispensables en microservices. Je cite Loki et Tempo (logs et traces côté Grafana) et OpenTelemetry (le standard d'instrumentation) sans m'y attarder : on y revient au module 16.

Pourquoi les métriques d'abord. Une métrique coûte quelques octets par point. Un log en coûte des centaines. Sur une flotte de cent serveurs, les métriques tiennent sur un disque de portable ; les logs, non. Et surtout : c'est sur les métriques qu'on alerte.

Anecdote — le Black Friday. Un site e-commerce, un vendredi de novembre. Tous les voyants infra au vert : CPU tranquille, mémoire tranquille, réseau tranquille. Chiffre d'affaires de la journée : zéro. Un bug JavaScript sur le bouton « Payer ». Personne n'avait de métrique « nombre de commandes par minute ». Le serveur allait très bien ; le business, lui, était à l'arrêt. C'est pour ça que notre boutique fil rouge expose `shop_orders_total` et `shop_revenue_euros_total` en plus de ses métriques HTTP. Surveiller la machine ne suffit jamais.

Pull contre push. Les outils historiques (Nagios, Zabbix, Graphite avec collectd) reposent souvent sur un agent qui *pousse* ses données vers un serveur central. Prometheus fait l'inverse : il va *chercher* (scraper) les métriques sur chaque cible en HTTP, à intervalle fixe.

Analogie du livreur de pizza contre le buffet : avec le push, si mille serveurs ont un souci en même temps, mille livreurs sonnent chez vous en même temps ; le monitoring tombe précisément au moment où on en a besoin. Avec le pull, Prometheus est le client du buffet : il se sert à son rythme, et si une cible ne répond pas, c'est une information (`up == 0`), pas un silence.

Autres avantages concrets du pull :

- On sait immédiatement quand une cible est morte.
- Une cible expose ses métriques en HTTP, on peut les lire avec un simple `curl` . Débogage trivial.
- Pas de configuration côté cible pour savoir « où envoyer ».

Le push garde un cas d'usage : les jobs éphémères (un batch de 3 secondes ne sera jamais scrapé). On verra la Pushgateway au TP 2.

Un peu d'histoire, vite. Né chez SoundCloud en 2012, inspiré de Borgmon (Google). Open source en 2015. Deuxième projet accueilli par la CNCF en 2016 après Kubernetes, gradué en 2018. Aujourd'hui, c'est le standard de fait : Kubernetes, Docker, la plupart des bases de données et des middlewares exposent nativement du format Prometheus. Version 3 sortie fin 2024 ; on travaille sur la 3.15 (septembre 2026).

Grafana est né en 2014 comme un fork de Kibana 3, avec une idée : afficher des séries temporelles de n'importe quelle source. Il ne stocke rien et ne collecte rien : il interroge Prometheus (et Loki, Tempo, SQL, Elasticsearch...) et dessine. Version 13 sortie en avril 2026 ; on utilise la 13.2.

Ce que je montre

Rien de technique ici. Je garde le dashboard rouge en arrière-plan.

Points de vigilance

Les stagiaires venus de Zabbix ou Centreon demandent souvent « mais alors, il n'y a pas d'agent ? ». Réponse : si, ça s'appelle un exporter, mais il est passif : il expose, il n'envoie pas. On le voit cet après-midi.

Module 2 — Architecture de Prometheus et place de Grafana 40 min

Objectif. Connaître les composants, le modèle de données et les quatre types de métriques. À la fin, un stagiaire doit pouvoir dessiner l'architecture au tableau.

Ce que je dis

Les composants. Je dessine au tableau, dans cet ordre :

1. Les **cibles** (targets) : tout ce qui expose une page `/metrics` en HTTP. Une application instrumentée, un exporter.
2. Le **serveur Prometheus** : un seul binaire Go qui scrape, stocke et évalue.
 - *Scraper* : toutes les 15 secondes (par défaut), il appelle chaque `/metrics` .
 - *Stocker* : dans sa base de séries temporelles locale, la TSDB.
 - *Évaluer* : des règles PromQL, pour pré-calculer (recording rules) ou pour alerter.

3. **Alertmanager** : un binaire séparé qui reçoit les alertes de Prometheus, les regroupe, les déduplique, les route vers Slack, Teams, PagerDuty, mail.
4. **Grafana** : interroge Prometheus en PromQL et dessine.
5. La **découverte de services** (service discovery) : au lieu de lister les cibles à la main, Prometheus les demande à Kubernetes, Consul, AWS, un fichier...

Analogie : Prometheus est un journaliste qui fait sa tournée toutes les 15 secondes et note tout dans son carnet. Alertmanager est le rédacteur en chef qui décide ce qui mérite un coup de fil à 3h du matin. Grafana est la maquette du journal.

Le modèle de données. C'est le concept le plus important de la journée.

Une *métrique* a un nom : `http_requests_total` . Une *série temporelle* est une métrique plus un jeu de labels : `http_requests_total{method="GET", route="/api/products", status="200"}` . Un *échantillon* (sample) est une valeur flottante et un timestamp en millisecondes.

Chaque combinaison de valeurs de labels crée une série distincte. C'est ce qui rend Prometheus puissant (on peut découper par n'importe quelle dimension) et c'est ce qui le tue quand on abuse (cardinalité, on y revient à la fin de la journée et au jour 3).

Les labels réservés : `job` (le nom du bloc de configuration), `instance` (l'adresse `hôte:port`), et tout ce qui commence par `__` est interne.

Les quatre types.

Type	Il fait quoi	Exemples	Ce qu'on en fait
Counter	Ne fait que monter (remis à zéro au redémarrage)	requêtes, erreurs, octets, commandes	<code>rate()</code> , <code>increase()</code> — jamais la valeur brute
Gauge	Monte et descend	mémoire, température, connexions actives, taille de file	valeur brute, <code>avg_over_time</code> , <code>max_over_time</code> , <code>deriv</code>
Histogram	Compte les observations par tranches (buckets)	latences, tailles de réponse	<code>histogram_quantile()</code> (p95, p99)
Summary	Quantiles calculés côté client	latences (ancienne école)	lecture directe des quantiles, non agrégeable

L'analogie de la voiture pour le Counter : « mon compteur affiche 150 000 km, est-ce que je roule vite ? » Aucune idée. Ce qui m'intéresse, c'est la dérivée : des km par heure. `rate()` transforme un compteur kilométrique en vitesse. Règle d'or : **un Counter ne se lit jamais brut.**

Histogram contre Summary : l'histogramme est calculé côté serveur, on peut additionner les buckets de dix instances et calculer un p95 global. Le Summary calcule ses quantiles dans l'application ; on ne peut pas faire la moyenne de deux p95. En 2026, on choisit l'histogramme sauf cas très particulier. Je mentionne les *native histograms* (buckets exponentiels automatiques, bien plus précis pour moins de séries), stables depuis Prometheus 3.9 et activés par `scrape_native_histograms: true` ; on les voit au jour 2.

Le format d'exposition. Du texte, lisible par un humain :

```
# HELP http_requests_total Nombre total de requêtes HTTP reçues
# TYPE http_requests_total counter
http_requests_total{method="GET",route="/api/products",status="200"} 51.0
```

Une ligne `HELP` , une ligne `TYPE` , puis une ligne par série. C'est tout. C'est pour ça que tout le monde l'a adopté : n'importe quel langage peut produire ça avec un `printf` .

La TSDB en deux mots. Les échantillons arrivent dans un bloc en mémoire (le *head*), sont journalisés dans un WAL (write-ahead log) pour survivre à un crash, puis toutes les deux heures un bloc immuable est écrit sur disque. Compression très efficace : un échantillon pèse en moyenne 1 à 2 octets. Rétention par défaut : 15 jours. On rentre dans le détail au jour 3.

Où est Grafana dans tout ça. Grafana ne voit jamais les cibles. Il ne connaît que Prometheus, à qui il envoie des requêtes PromQL via l'API HTTP `/api/v1/query_range` . Si Grafana affiche « No data », le réflexe est de tester la même requête directement dans Prometheus pour savoir qui est en cause.

Ce que je montre

- `http://localhost:5001/metrics` : je fais défiler, je pointe un `# TYPE ... counter` , un `gauge` , un `histogram` avec ses buckets `le=` , et le `summary` avec `_count` et `_sum` .
- `http://localhost:9090` → **Status** → **Target health** : la liste des cibles, chacune avec son dernier scrape.

The screenshot shows the Prometheus 3 Status -> Target health page. The interface includes a top navigation bar with 'Prometheus', 'Query', 'Alerts', and 'Status -> Target health'. Below the navigation bar, there are filters for 'Select scrape pool', 'Filter by target health', and 'Filter by endpoint or labels'. A blue banner indicates 'Hiding pools with no matching targets' and 'Hiding 1 empty pools due to filters or no targets. Show empty pools'. The main content area displays two jobs: 'blackbox-http' and 'node'. The 'blackbox-http' job has 4 targets, all in 'UP' state. The 'node' job has 1 target, also in 'UP' state. Each target row shows the Endpoint, Labels, Last scrape time, and State.

Endpoint	Labels	Last scrape	State
http://blackbox-exporter:9115/probe	instance="https://prometheus.io" job="blackbox-http"	373ms ago 6ms	UP
http://blackbox-exporter:9115/probe	instance="http://shop-api-1:5000/health" job="blackbox-http"	9.756s ago 4ms	UP
http://blackbox-exporter:9115/probe	instance="http://shop-api-2:5000/health" job="blackbox-http"	8.395s ago 3ms	UP
http://blackbox-exporter:9115/probe	instance="http://grafana:3000/api/health" job="blackbox-http"	4.339s ago 2ms	UP
node			

Prometheus 3 — Status → Target health, une fois les jobs du jour 1 ajoutés

- Dans la barre de requête : `up` , puis `http_requests_total` . « Ça, c'est PromQL. On y passe la journée de demain. »

Points de vigilance

Le Summary de notre application Python n'expose pas de quantiles (la bibliothèque Python ne les implémente pas, uniquement `_count` et `_sum`). C'est un piège réel et un bon argument pour l'histogramme. Pour montrer un vrai Summary avec quantiles, je prends `go_gc_duration_seconds` exposé par Prometheus lui-même.

Module 3 — Installation et prise en main 50 min

Objectif. Savoir comment on installe Prometheus et Grafana « en vrai », et maîtriser l'environnement du lab.

Ce que je dis

Rappels d'installation. Cinq façons d'installer Prometheus, par ordre de fréquence en 2026 :

1. **Kubernetes avec l'opérateur** (kube-prometheus-stack, Helm) : le cas majoritaire en production. Les cibles sont découvertes automatiquement, la configuration passe par des objets `ServiceMonitor` et `PrometheusRule`.
2. **Conteneur Docker** : `docker run -p 9090:9090 -v ./prometheus.yml:/etc/prometheus/prometheus.yml prom/prometheus:v3.15.0`. C'est notre cas.
3. **Binaire** : une archive tar.gz sur GitHub, un seul exécutable, `./prometheus --config.file=prometheus.yml`. Idéal pour comprendre : pas de magie, un binaire, un fichier YAML, un dossier `data/`.
4. **Paquet distribution** (apt, dnf) : souvent en retard de plusieurs versions, je déconseille.
5. **Managé** : Grafana Cloud, Amazon Managed Prometheus, Google Cloud Managed Service for Prometheus, ou un stockage compatible (Mimir, VictoriaMetrics, Thanos). On en parle au jour 3.

Même chose pour Grafana : Helm, Docker (`grafana/grafana:13.2.1`), paquet `.deb` / `.rpm` (Grafana Labs maintient ses propres dépôts, à jour), ou Grafana Cloud.

Les fichiers et dossiers qui comptent :

Prometheus	Grafana
<code>prometheus.yml</code> : la configuration	<code>grafana.ini</code> (ou <code>custom.ini</code> , ou variables <code>GF_*</code>)
<code>--storage.tsdb.path</code> (défaut <code>data/</code>) : la base	<code>/var/lib/grafana</code> : la base SQLite <code>grafana.db</code> , les plugins
<code>--storage.tsdb.retention.time</code> (défaut 15d)	<code>/etc/grafana/provisioning/</code> : datasources, dashboards, alerting en YAML
<code>--web.enable-lifecycle</code> : autorise le reload par HTTP	port 3000, compte <code>admin</code>
port 9090	

Pourquoi Docker pour la formation. Parce qu'on a huit services à faire tourner et que je veux que tout le monde ait exactement la même chose. Le `docker-compose.yml` est lisible : on l'ouvre ensemble.

Ce que je montre

J'ouvre `docker-compose.yml` et je commente service par service, sans entrer dans les détails : les deux instances de la boutique, le générateur de trafic, Redis et son exporter, la boîte de réception, Prometheus avec ses flags, Alertmanager, les exporters, Grafana avec son provisioning. Je m'arrête sur les flags de Prometheus :

```
command:
- --config.file=/etc/prometheus/prometheus.yml
- --storage.tsdb.path=/prometheus
- --storage.tsdb.retention.time=7d
- --web.enable-lifecycle           # POST /-/reload
- --web.enable-admin-api          # snapshots (Jour 3)
- --web.enable-remote-write-receiver
- --web.external-url=http://localhost:9090
```

Puis `docker compose exec prometheus prometheus --help | head -60` pour montrer qu'il y a une centaine de flags et que ceux-là sont les seuls qu'on touche en général.

Exercice 1.1 — Démarrer et vérifier 10 min

Énoncé (guide stagiaire).

1. Lancez la stack : `./lab.sh up` (ou `.\lab.ps1 up`, ou créez votre Codespace).
2. Vérifiez que tous les conteneurs sont Up : `./lab.sh status`.
3. Ouvrez Prometheus, Grafana (admin / formation), Alertmanager et l'Inbox dans quatre onglets.
4. Dans Prometheus, **Status** → **Target health** : combien de cibles sont UP ? Notez-les.

Corrigé. Trois cibles au démarrage : `prometheus` (localhost:9090) et les deux instances `shop-api`. Le Node Exporter tourne mais n'est pas encore scrapé : c'est le TP 1.

Ce que je vérifie. Que tout le monde a bien 3/3 UP. Erreurs classiques : port 3000 déjà pris (un autre Grafana, ou un serveur de dev Node) ; Docker Desktop pas démarré ; sur Windows, le script bloqué par la politique d'exécution.

Exercice 1.2 — Lire une page /metrics 10 min

Énoncé. Ouvrez `http://localhost:5001/metrics` (ou le port 5001 dans l'onglet Ports).

1. Trouvez une métrique de chaque type : counter, gauge, histogram, summary. Notez leur nom.
2. Pour `http_request_duration_seconds`, combien de buckets ? Quelle est la borne du dernier ?
3. Quelle est la version de l'application ? Où est-elle stockée ?
4. Que vaut `shop_revenue_euros_total` ? Rechargez la page : elle a bougé ?

Corrigé.

1. Counter : `http_requests_total`, `shop_orders_total`, `shop_revenue_euros_total`. Gauge : `shop_cart_items`, `shop_stock_units`, `http_requests_in_progress`. Histogram : `http_request_duration_seconds` (`_bucket`, `_sum`, `_count`). Summary : `shop_payment_duration_seconds` (`_count`, `_sum`, pas de quantiles en Python).
2. 11 buckets explicites, de `le="0.005"` à `le="10.0"`, plus `le="+Inf"`. Chaque bucket est cumulatif : `le="0.5"` compte toutes les requêtes de moins de 500 ms.

3. `shop_app_info{instance_name="shop-api-1",version="1.4.2"} 1.0` . La valeur est toujours 1 ; l'information est dans les labels. C'est le pattern *info metric*, on l'utilisera au jour 2 pour une jointure.
4. Elle monte à chaque commande. Un counter, donc.

Je fais remarquer les métriques `python_*` et `process_*` : la bibliothèque cliente les ajoute gratuitement (mémoire du processus, GC, descripteurs de fichiers).

Exercice 1.3 — L'interface de Prometheus 10 min

Énoncé.

1. **Status → Configuration** : retrouvez le `scrape_interval` global.
2. **Status → Runtime & build information** : quelle version de Prometheus ? Depuis combien de temps tourne-t-il ?
3. **Status → TSDB status** : combien de séries en mémoire ? Quelle métrique a le plus de séries ?
4. Onglet **Query** : tapez `up` et exécutez. Puis passez en onglet **Graph**.

Corrigé. `scrape_interval: 15s` . Version 3.15.0. Environ 2 000 séries après quelques minutes ; la métrique la plus « lourde » est `http_request_duration_seconds_bucket` (12 buckets × routes × 2 instances). `up` renvoie 3 séries à 1.

Je fais remarquer la nouvelle interface de Prometheus 3 (l'onglet *Explain* qui décompose une requête, le mode *Table / Graph*, la complétion dans la barre de requête).

Exercice 1.4 — Premières requêtes 10 min

Énoncé. Dans l'onglet Query :

1. `http_requests_total` : combien de séries ?
2. Ne gardez que les requêtes de `shop-api-2` (label `instance`).
3. Ne gardez que les erreurs (codes 4xx et 5xx) : label `status` , expression régulière.
4. Combien de requêtes `POST` sur `/api/checkout` depuis le démarrage de `shop-api-1` ?

Corrigé.

1. Une quarantaine, une vingtaine par instance (ça dépend des routes déjà appelées) : chaque combinaison méthode × route × code × instance est une série.
2. `http_requests_total{instance="shop-api-2:5000"}`
3. `http_requests_total{status=~"4..|5.."}`
4. `http_requests_total{instance="shop-api-1:5000", method="POST", route="/api/checkout"}` : une valeur par code (201 et éventuellement 502).

Je ne vais pas plus loin : la syntaxe complète, c'est demain. L'important aujourd'hui, c'est `{label="valeur"}` , `!=` et `=~` .

Module 4 — Configuration de Prometheus 40 min

Objectif. Lire et modifier `prometheus.yml` en sécurité : valider, recharger à chaud, découvrir des cibles autrement qu'à la main, réécrire des labels.

Ce que je dis

Anatomie du fichier. Quatre blocs :

```
global:                # les réglages par défaut
  scrape_interval: 15s
  evaluation_interval: 15s
  external_labels:      # ajoutés quand Prometheus parle à l'extérieur (Alertmanager,
remote_write)
  cluster: formation
rule_files:             # règles d'enregistrement et d'alerte
  - rules/*.yaml
alerting:               # où sont les Alertmanager
  alertmanagers:
    - static_configs:
      - targets: ["alertmanager:9093"]
scrape_configs:         # la liste des jobs
  - job_name: shop-api
    static_configs:
      - targets: ["shop-api-1:5000", "shop-api-2:5000"]
    labels:
      env: formation
```

Un `job` regroupe des cibles de même nature. Chaque cible reçoit automatiquement `job` et `instance`. Les labels ajoutés sous `static_configs` s'appliquent à toutes les cibles du bloc.

Le rythme. `scrape_interval` : trop court, on charge les cibles et le disque ; trop long, on rate les pics. 15 s est le standard, 30 s ou 60 s pour les gros parcs, 5 s pour un besoin précis. On peut le surcharger job par job. Règle : la fenêtre de `rate()` (jour 2) doit faire au moins 4 fois le `scrape_interval`.

Recharger sans redémarrer. Deux méthodes : `kill -HUP <pid>` ou, si `--web.enable-lifecycle` est actif, `curl -X POST http://localhost:9090/-/reload`. Si le fichier est invalide, Prometheus garde l'ancienne configuration et le dit dans ses logs, et la métrique `prometheus_config_last_reload_successful` passe à 0. On alertera dessus au jour 3.

Valider avant. `promtool check config prometheus.yml`. Toujours. C'est le `nginx -t` de Prometheus. Dans le lab : `./lab.sh check`.

La découverte de services. Lister les cibles à la main ne tient pas au-delà de dix serveurs.

Prometheus sait interroger : Kubernetes (`kubernetes_sd_configs` , le plus utilisé), Consul, DNS, EC2/Azure/GCE, Docker, et le plus simple de tous, un fichier (`file_sd_configs`) que n'importe quel script peut écrire. Le fichier est relu automatiquement, sans reload. C'est le pont idéal avec un outil existant (CMDDB, Ansible, Terraform).

Le relabeling. Entre la découverte d'une cible et son scrape, Prometheus passe la cible dans une chaîne de règles `relabel_configs` qui peuvent renommer, filtrer, réécrire. C'est le videur à l'entrée de la boîte de nuit : il regarde les labels (qui commencent par `__meta__` pour la découverte) et décide qui entre et sous quel nom. `metric_relabel_configs` fait la même chose après le scrape, sur chaque série : c'est là qu'on jette les métriques inutiles avant qu'elles ne coûtent du disque. On en voit un usage concret avec le Blackbox Exporter au TP 2.

Ce que je montre

- `./lab.sh check` puis je casse volontairement l'indentation d'une ligne, `./lab.sh check` refuse, je répare.
- `./lab.sh reload`, puis dans Prometheus **Status** → **Configuration** pour montrer que c'est bien la version rechargée.
- `docker compose logs --tail 20 prometheus` pour montrer la ligne `Completed loading of configuration file`.

Exercice 1.5 — Changer le rythme 10 min

Énoncé.

1. Pour le job `shop-api` uniquement, passez le `scrape_interval` à 5 s.
2. Validez la configuration, rechargez.
3. Vérifiez dans **Status** → **Target health** (colonne *Last scrape*) et avec la requête `prometheus_target_interval_length_seconds{quantile="0.99"}`.

Corrigé.

```
- job_name: shop-api
  scrape_interval: 5s
  static_configs:
    - targets: ["shop-api-1:5000", "shop-api-2:5000"]
```

`./lab.sh check && ./lab.sh reload`. Dans Target health, *Last scrape* ne dépasse plus 5 s. La requête renvoie une série par intervalle configuré : on voit apparaître `interval="5s"`. Je demande ensuite de **remettre 15 s** : on ne veut pas fausser les exercices `rate()` de demain.

Ce que je vérifie. Que le reload a été fait (beaucoup oublient et attendent que ça change tout seul).

Exercice 1.6 — Casser pour comprendre 5 min

Énoncé.

1. Introduisez une erreur dans `prometheus.yml` (un `:` en trop, un tiret manquant).
2. Rechargez **sans** valider. Que se passe-t-il ? Regardez les logs et la métrique `prometheus_config_last_reload_successful`.
3. Réparez, rechargez, vérifiez que la métrique repasse à 1.

Corrigé. Le `curl -X POST /-/reload` renvoie une erreur HTTP 500 avec le message de parsing. Prometheus continue de tourner avec l'ancienne configuration. La métrique vaut 0 tant que le fichier est invalide. Message à faire passer : Prometheus est conservateur, mais une configuration cassée qui traîne, personne ne s'en rend compte sans alerte.

Exercice 1.7 — Découverte par fichier 10 min

Énoncé. La boîte de réception (`inbox`) expose aussi une page `/metrics` sur le port 8080. Ajoutez-la à Prometheus **sans** la lister dans `prometheus.yml` :

1. Ajoutez un job `file-sd` qui lit tous les fichiers `targets/*.yaml`.

2. Créez `prometheus/targets/extra.yml` avec la cible `inbox:8080` et un label `tier: outils`.
3. Rechargez une fois (pour le nouveau job), puis vérifiez que la cible apparaît : les fichiers de cibles, eux, sont relus toutes les 30 s sans reload.

Corrigé.

```
- job_name: file-sd
  file_sd_configs:
    - files: ["targets/*.yml", "targets/*.json"]
      refresh_interval: 30s
```

`prometheus/targets/extra.yml` :

```
- targets: ["inbox:8080"]
  labels:
    tier: outils
```

Un reload est nécessaire pour le nouveau job (une fois), pas pour les fichiers de cibles qui sont relus toutes les 30 s. Dans Target health, le job `file-sd` apparaît avec la cible `inbox:8080` UP, et `inbox_messages_received_total` est disponible. On l'utilisera au jour 3 pour compter les notifications reçues.

Ce que je vérifie. Le chemin des fichiers est relatif au dossier de `prometheus.yml` (`/etc/prometheus` dans le conteneur, donc `prometheus/targets/` sur la machine). Erreur classique : écrire un chemin absolu de la machine hôte.

Exercice 1.8 — Relabeling (bonus, 10 min, après le TP 1)

Énoncé. Le Node Exporter expose ses propres métriques internes (`go_*`, `process_*`, `promhttp_*`) en plus des métriques de la machine. Ajoutez un second job `node-light` qui scrape `node-exporter:9100` en jetant ces métriques internes. Comparez le nombre de séries des deux jobs avec `count by (job) ({job=~"node.*"})`.

Corrigé.

```
- job_name: node-light
  static_configs:
    - targets: ["node-exporter:9100"]
  metric_relabel_configs:
    - source_labels: [__name__]
      regex: "go_.*|process_.*|promhttp_.*"
      action: drop
```

Le job complet a une quarantaine de séries de plus. Sur un parc de cinq cents machines, ça fait vingt mille séries économisées. C'est le genre de règle qu'on met en place dès le début et qu'on oublie ensuite. Une fois la comparaison faite, je fais **retirer** `node-light` : deux jobs sur la même cible dupliquent toutes les séries, et les dashboards de demain afficheraient tout en double. En vrai, on met le `metric_relabel_configs` directement sur le job `node`.

Module 5 — Les exporters 30 min

Objectif. Comprendre ce qu'est un exporter, savoir en choisir un et le configurer.

Ce que je dis

Le problème. Prometheus parle HTTP et ne comprend que son format texte. Le noyau Linux parle `/proc` et `/sys`. PostgreSQL parle SQL. Un switch parle SNMP. Un exporter est un adaptateur : un petit programme qui interroge le système d'un côté et expose une page `/metrics` de l'autre. L'adaptateur de prise universel.

Les incontournables.

Exporter	Pour quoi	Port habituel
node_exporter	Linux : CPU, mémoire, disque, réseau, systemd	9100
windows_exporter	Windows : idem, plus IIS, MSSQL, AD	9182
blackbox_exporter	Sondes externes : HTTP, TCP, ICMP, DNS, TLS	9115
cAdvisor / kube-state-metrics	Conteneurs / objets Kubernetes	8080
postgres_exporter, mysqld_exporter, redis_exporter, mongodb_exporter	Bases de données	9187, 9104, 9121, 9216
snmp_exporter	Équipements réseau	9116
pushgateway	Pas un exporter : un relais pour les batchs	9091

Le catalogue officiel (prometheus.io/docs/instrumenting/exporters) en liste des centaines. Avant d'en écrire un, on vérifie qu'il n'existe pas. Et de plus en plus de logiciels exposent nativement du Prometheus : Traefik, HAProxy, Kubernetes, etcd, RabbitMQ, Kafka, Grafana lui-même.

Node Exporter en détail. Une soixantaine de *collectors*, une trentaine activés par défaut (cpu, meminfo, filesystem, netdev, loadavg, diskstats, uname, time...). On active ou désactive avec `--collector.<nom>` / `--no-collector.<nom>`. Le collector `textfile` lit des fichiers `.prom` dans un dossier : n'importe quel script cron peut y déposer des métriques. C'est la Pushgateway du pauvre, et elle est souvent préférable.

Le piège Docker. Un Node Exporter lancé dans un conteneur sans précaution voit le conteneur : un processeur virtuel, quelques mégaoctets, un système de fichiers overlay. Il faut lui monter `/proc`, `/sys` et `/` de l'hôte en lecture seule et lui dire où les trouver (`--path.procfs`, `--path.sysfs`, `--path.rootfs`). C'est ce que fait notre compose. Et sur Docker Desktop (Mac, Windows), « l'hôte » est la VM Linux de Docker, pas votre machine. En production sur Linux, on l'installe plutôt en paquet ou en binaire directement sur l'hôte.

Blackbox Exporter. Tous les autres exporters disent « je vais bien » de l'intérieur. Le blackbox teste de l'extérieur, comme un client : la page répond-elle en 200 ? Le certificat expire-t-il bientôt ? Le port 5432 accepte-t-il une connexion ? Il fonctionne à l'envers des autres : Prometheus l'appelle sur `/probe?module=http_2xx&target=https://...`, et c'est là que le relabeling devient indispensable.

Pushgateway. Pour les jobs trop courts pour être scrapés. Le job pousse ses métriques, la Pushgateway les garde et Prometheus les scrape. Attention : elle ne les oublie jamais (pas de TTL) et elle transforme le pull en push avec tous ses défauts. Uniquement pour les batchs, jamais pour des services.

Ce que je montre

- `http://localhost:9100/metrics` : `node_cpu_seconds_total` avec ses modes, `node_memory_MemAvailable_bytes`, `node_filesystem_avail_bytes`, `node_uname_info`.
- `http://localhost:9115` : la page du blackbox avec l'historique des sondes (vide pour l'instant), puis `http://localhost:9115/probe?module=http_2xx&target=http://shop-api-1:5000/health` pour montrer `probe_success 1` et `probe_duration_seconds`.

TP 1 — Node Exporter : de la machine à Prometheus 60 min

Objectif. Le cas pratique du programme : ajouter et configurer un Node Exporter, en tirer les indicateurs classiques d'un serveur Linux.

Mise en situation (guide stagiaire). Vous venez de recevoir un serveur. Avant d'y déployer quoi que ce soit, vous voulez ses signes vitaux dans Prometheus : CPU, mémoire, disque, réseau, uptime. Et vous voulez pouvoir y ajouter vos propres indicateurs avec un simple script.

Partie 1 — Brancher l'exporter 10 min

Énoncé.

1. Dans `prometheus/prometheus.yml`, ajoutez un job `node` qui scrape `node-exporter:9100`.
2. Validez, rechargez, vérifiez la cible dans Target health.
3. Requête : `node_uname_info`. Quel noyau ? Quel nom de machine ?

Corrigé.

```
- job_name: node
  static_configs:
    - targets: ["node-exporter:9100"]
```

`node_uname_info{nodename="...", release="6.x..."}`. Sur Docker Desktop, le `nodename` est celui de la VM (par exemple `docker-desktop`). Sur Codespaces, celui du conteneur de dev.

Partie 2 — Les indicateurs de base 15 min

Énoncé. Écrivez une requête pour chacun des indicateurs suivants (on accepte des approximations, la syntaxe complète est pour demain, les fonctions nécessaires sont données) :

1. Uptime en secondes : `time()` et `node_boot_time_seconds`.

2. Mémoire disponible en pourcentage : `node_memory_MemAvailable_bytes` , `node_memory_MemTotal_bytes` .
3. Espace disque utilisé en % sur / : `node_filesystem_avail_bytes` , `node_filesystem_size_bytes` , label `mountpoint` .
4. Charge moyenne 1 min : `node_load1` .
5. CPU utilisé en % (la formule est donnée, à recopier et à comprendre) : `100 * (1 - avg(rate(node_cpu_seconds_total{mode="idle"}[5m])))` .

Corrigé.

1. `time() - node_boot_time_seconds`
2. `100 * node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes`
3. `100 * (1 - node_filesystem_avail_bytes{mountpoint="/" } / node_filesystem_size_bytes{mountpoint="/" })`
4. `node_load1` (et je fais comparer à `count(node_cpu_seconds_total{mode="idle"})` , le nombre de cœurs : une charge de 2 sur 2 cœurs, c'est plein).
5. La formule : `node_cpu_seconds_total` est un counter par cœur et par mode (secondes passées dans chaque mode). `rate(...[5m])` donne la fraction du temps passée en `idle` sur 5 min, par cœur. `avg` fait la moyenne des cœurs. `1 - idle` = occupé. `* 100` = pourcentage. Je passe cinq minutes là-dessus : tout le monde la recopie, peu de gens la comprennent la première fois.

Je fais lancer `./lab.sh chaos cpu 120` et observer la courbe monter sur l'onglet Graph.

Partie 3 — Configurer les collectors 10 min

Énoncé.

1. Le collector `processes` (nombre de processus, threads) est désactivé par défaut. Activez-le en ajoutant `--collector.processes` aux `command` du service `node-exporter` dans `docker-compose.yml` , puis `docker compose up -d node-exporter` .
2. Vérifiez l'apparition de `node_processes_state` et `node_processes_threads` .
3. Désactivez un collector inutile pour vous (par exemple `--no-collector.arp`) et vérifiez que `node_arp_entries` disparaît de `/metrics` .

Corrigé.

```
command:
  - --path.procfs=/host/proc
  - --path.sysfs=/host/sys
  - --path.rootfs=/rootfs
  - --collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc|rootfs/var/lib/docker/.)($$|/)
  - --collector.textfile.directory=/textfile
  - --collector.processes
  - --no-collector.arp
```

Remarque à faire : dans une chaîne compose, `$$` échappe le `$` de la regex. Ce n'est pas un piège du Node Exporter, c'est un piège de compose. Et `node_arp_entries` disparaît de la page `/metrics` mais reste dans Prometheus pendant quelques minutes (les séries périmées, on en parle au jour 3).

Partie 4 — Le textfile collector 10 min

Énoncé. Vous avez un script de sauvegarde en cron. Vous voulez savoir quand il a tourné pour la dernière fois.

1. Créez `node-exporter/textfile/backup.prom` avec une gauge `backup_last_run_timestamp_seconds` (valeur : le timestamp actuel, `date +%s`) et une gauge `backup_files_total` (valeur au choix).
2. Vérifiez sur `http://localhost:9100/metrics` puis dans Prometheus.
3. Calculez « il y a combien de temps » : `time() - backup_last_run_timestamp_seconds`.

Corrigé.

```
# HELP backup_last_run_timestamp_seconds Dernière exécution du script de sauvegarde
# TYPE backup_last_run_timestamp_seconds gauge
backup_last_run_timestamp_seconds 1790404000
# HELP backup_files_total Fichiers sauvegardés lors de la dernière exécution
# TYPE backup_files_total gauge
backup_files_total 1234
```

Sur Mac/Linux : `echo "backup_last_run_timestamp_seconds $(date +%s)" > node-exporter/textfile/backup.prom`. Sur Windows PowerShell, attention aux fins de ligne : `Set-Content` écrit du CRLF, que le collector refuse. Il faut forcer le LF : `[IO.File]::WriteAllText("node-exporter/textfile/backup.prom", "backup_last_run_timestamp_seconds $([DateTimeOffset]::UtcNow.ToUnixTimeSeconds()) n")``.

Point d'attention : le fichier doit se terminer par un saut de ligne, en LF uniquement, et une erreur de format fait échouer tout le fichier (métrique `node_textfile_scrape_error` à 1). Je montre la métrique.

Anecdote — les sauvegardes fantômes. Chez un client, un script de sauvegarde écrivait « OK » dans un log depuis dix-huit mois. Personne ne lisait le log. Le jour où on a eu besoin de restaurer, la dernière sauvegarde valide datait de dix-huit mois : le montage NFS avait disparu, le script écrivait dans le vide. Une gauge `backup_last_success_timestamp_seconds` et une alerte « plus de 24 h » auraient coûté dix minutes. On écrit cette alerte au jour 3.

Partie 5 — Vue d'ensemble 15 min

Énoncé.

1. Quel est le disque (`mountpoint`) le plus rempli ? Utilisez `topk(1, ...)` sur la requête de la partie 2.
2. Combien d'octets par seconde entrent sur les interfaces réseau (hors `lo`) ?
`rate(node_network_receive_bytes_total{device!="lo"}[5m])`.
3. Dans Grafana, **Explore** (menu de gauche) : collez la requête CPU et regardez-la sur les 30 dernières minutes. Repérez le moment où vous avez lancé le chaos CPU.
4. Bonus : importez le dashboard communautaire *Node Exporter Full* (Dashboards → New → Import, ID 1860) et regardez ce que vous saurez construire demain. Attention, il a besoin d'un accès à grafana.com pour l'import par ID ; sinon, on s'en passe.

Corrigé. `topk(1, 100 * (1 - node_filesystem_avail_bytes / node_filesystem_size_bytes))`. Le dashboard 1860 est parfait pour montrer la puissance de Grafana... et ses excès : des dizaines de panneaux, personne ne les lit. Demain on construira le nôtre, avec dix panneaux qui répondent à de

vraies questions.

Ce que je vérifie. Que le job `node` est bien dans le `prometheus.yml` de tout le monde : les dashboards de demain en dépendent.

Module 6 — Instrumenter son application 30 min

Objectif. Savoir ajouter une métrique dans du code, choisir son type et ses labels, éviter l'explosion de cardinalité.

Ce que je dis

Whitebox contre blackbox. Le Node Exporter et le Blackbox regardent l'application de l'extérieur. Instrumenter, c'est mettre des sondes à l'intérieur : on sait combien de commandes ont été passées, combien de paiements ont échoué, combien de temps prend l'appel à la banque. C'est la seule façon d'avoir des métriques *métier*.

Les bibliothèques clientes. Officielles : Go, Java/JVM (et Micrometer pour Spring), Python, Ruby, Rust. Communautaires pour tout le reste (.NET avec prometheus-net, Node.js avec prom-client, PHP...). Toutes font la même chose : on déclare des objets Counter/Gauge/ Histogram/Summary, on les met à jour dans le code, et la bibliothèque expose `/metrics`. Pas de base de données, pas de fichier : tout est en mémoire dans le processus, ce qui explique le compteur remis à zéro au redémarrage.

Le code de notre boutique (`apps/shop-api/app.py`) :

```
HTTP_REQUESTS = Counter("http_requests_total", "Nombre total de requêtes HTTP reçues",
                        ["method", "route", "status"])
HTTP_DURATION = Histogram("http_request_duration_seconds", "Durée de traitement",
                          ["route"], buckets=(0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1, 2.5,
7, 10))
ORDERS = Counter("shop_orders_total", "Commandes validées", ["payment_method"])
STOCK = Gauge("shop_stock_units", "Unités en stock par produit", ["product"])

# dans le code
HTTP_REQUESTS.labels(method="POST", route="/api/checkout", status="201").inc()
HTTP_DURATION.labels(route="/api/checkout").observe(0.083)
ORDERS.labels(payment_method="card").inc()
STOCK.labels(product="clavier").set(84)
```

Les conventions de nommage. Elles comptent, parce que PromQL et les dashboards communautaires les supposent :

- `snake_case`, préfixe du domaine : `http_`, `shop_`, `node_`, `process_`.
- L'unité en suffixe, en unités de base : `_seconds`, `_bytes`, `_total` pour les counters. Pas de millisecondes, pas de kilo-octets : Grafana convertit.
- Un nom décrit une chose mesurée, les labels la découpent. `http_requests_total{status="500"}`, pas `http_errors_500_total`.
- Une gauge d'information : `_info` avec valeur 1.

Les buckets d'histogramme. Par défaut : de 5 ms à 10 s, adaptés à du web. Pour un batch qui dure des minutes ou une requête SQL en microsecondes, on les redéfinit. Un p95 ne peut pas être plus précis que les buckets : si tout tombe entre `le="0.5"` et `le="1.0"`, le p95 dira « quelque part entre 500 ms et 1 s ». C'est l'argument des native histograms.

RED et USE. Deux méthodes pour ne rien oublier :

- Pour un *service* : **Rate** (débit), **Errors**, **Duration**. Nos trois métriques HTTP.
- Pour une *ressource* (CPU, disque, file) : **Utilisation**, **Saturation**, **Errors**. Les *golden signals* de Google SRE ajoutent la saturation aux trois de RED.

La cardinalité. Chaque valeur distincte d'un label crée une série. Une série coûte de la mémoire (quelques ko dans le head) et de l'index. Quelques règles :

- `method` (5 valeurs), `status` (10), `route` (50) : parfait.
- `user_id`, `session_id`, `request_id`, une adresse IP client, un email : interdit. Un million d'utilisateurs = un million de séries = Prometheus qui meurt.
- `route` doit être le *pattern* (`/api/products/<product>`), jamais l'URL réelle (`/api/products/clavier`). C'est ce que fait notre middleware avec `request.url_rule.rule`.
- Une recherche `?q=` ne doit jamais devenir un label. Notre route `/api/search` le dit en commentaire.

Anecdote — le label qui a tué Prometheus. Une équipe avait ajouté `customer_id` sur ses métriques HTTP « pour pouvoir filtrer par client ». Trente mille clients, cinquante routes, dix codes HTTP : quinze millions de séries. Prometheus consommait 60 Go de RAM et redémarrait toutes les heures. Solution : retirer le label, et pour le besoin réel (un client se plaint), passer par les logs ou les traces, qui sont faits pour les données à forte cardinalité.

Ce que je montre

`apps/shop-api/app.py` dans l'éditeur : les déclarations en haut, le middleware `after_request` qui observe chaque requête, la route `/api/checkout` qui incrémente les compteurs métier. Je pointe le `TODO` du TP 2.

TP 2 — Instrumentation et services tiers 60 min

Objectif. Ajouter une métrique métier dans le code, brancher un exporter tiers, des sondes externes et un batch. À la fin, `prometheus.yml` compte sept jobs.

Mise en situation. Le directeur commercial veut savoir quelles fiches produit sont les plus consultées. L'équipe infra veut surveiller Redis. Le support veut être prévenu si le site est inaccessible depuis l'extérieur. Et il y a ce batch de sauvegarde nocturne...

Partie 1 — Une métrique métier dans le code 15 min

Énoncé.

1. Dans `apps/shop-api/app.py`, déclarez un Counter `shop_product_views_total` avec un label `product` (cherchez le `TODO`).
2. Incrémentez-le dans la route `/api/products/<product>`.
3. Reconstituez et redémarrez les deux instances : `docker compose up -d --build shop-api-1 shop-api-2`.
4. Vérifiez sur `/metrics`, puis dans Prometheus : quel produit est le plus consulté ? `topk(1, sum by (product) (shop_product_views_total))`.

Corrigé.

```
PRODUCT_VIEWS = Counter(  
    "shop_product_views_total",  
    "Consultations de fiche produit",  
    ["product"],  
)
```

Dans `product_detail`, avant `redis_incr(...)` :

```
PRODUCT_VIEWS.labels(product=product).inc()
```

Le `clavier` gagne (le générateur de trafic le favorise). Question à poser : « pourquoi l'incrément est-il placé *après* le test `if product not in PRODUCTS` ? » Parce que sinon, n'importe qui pourrait créer des séries à l'infini en appelant `/api/products/nimportequoi`. Le label `product` n'est sûr que parce que ses valeurs sont bornées par le catalogue.

Ce que je vérifie. Le rebuild : `docker compose up -d` sans `--build` relance l'ancienne image et rien ne change. Sur Codespaces, le build prend 30 à 60 s.

Partie 2 — Un exporter tiers : Redis 10 min

Énoncé. La boutique stocke des compteurs dans Redis. Un `redis-exporter` tourne déjà sur le port 9121.

1. Ajoutez le job `redis` dans `prometheus.yml`. Validez, rechargez.
2. Requêtes : `redis_up`, `redis_connected_clients`, `rate(redis_commands_processed_total[5m])`.
3. Trouvez la commande Redis la plus utilisée : `topk(3, rate(redis_commands_total[5m]))`.

Corrigé.

```
- job_name: redis  
  static_configs:  
    - targets: ["redis-exporter:9121"]
```

`incrby` domine (chaque appel API incrémente un compteur ; `redis-py` envoie `INCRBY`). Remarque : l'exporter est configuré par variable d'environnement `REDIS_ADDR` dans le compose ; c'est le pattern habituel, un exporter par instance de service, colocalisé (sidecar en Kubernetes).

Partie 3 — Sondes externes avec Blackbox 15 min

Énoncé. Vous voulez vérifier depuis l'extérieur que :

- `http://shop-api-1:5000/health` et `http://shop-api-2:5000/health` répondent 2xx,
 - `http://grafana:3000/api/health` répond,
 - `https://prometheus.io` est joignable (accès Internet nécessaire ; sinon, ignorez).
1. Ajoutez un job `blackbox-http` avec `metrics_path: /probe`, le paramètre `module: [http_2xx]`, les quatre cibles, et le relabeling qui fait passer la cible en paramètre `target` et redirige l'adresse vers `blackbox-exporter:9115`. Le squelette est dans le guide stagiaire ; à vous de le compléter.
 2. Requêtes : `probe_success`, `probe_duration_seconds`, `probe_http_status_code`.
 3. Arrêtez `shop-api-2` (`docker compose stop shop-api-2`) et observez `probe_success` et `up`. Redémarrez-le.
 4. Bonus : `probe_ssl_earliest_cert_expiry - time()` pour `prometheus.io` : dans combien de jours le certificat expire-t-il ?

Corrigé.

```
- job_name: blackbox-http
  metrics_path: /probe
  params:
    module: [http_2xx]
  static_configs:
    - targets:
      - http://shop-api-1:5000/health
      - http://shop-api-2:5000/health
      - http://grafana:3000/api/health
      - https://prometheus.io
  relabel_configs:
    - source_labels: [__address__]
      target_label: __param_target
    - source_labels: [__param_target]
      target_label: instance
    - target_label: __address__
      replacement: blackbox-exporter:9115
```

Je décortique les trois règles au tableau, c'est le meilleur exemple de relabeling qui existe :

1. L'adresse de la cible (`__address__`, ce que Prometheus s'apprête à scraper) est copiée dans `__param_target`, ce qui ajoute `?target=...` à l'URL.
2. Cette même valeur devient le label `instance`, pour qu'on sache de quelle cible on parle.
3. Enfin, `__address__` est remplacée par l'adresse de l'exporter : c'est lui qu'on scrape.

Résultat : Prometheus appelle `http://blackbox-exporter:9115/probe?`

`module=http_2xx&target=http://shop-api-1:5000/health` et étiquette le résultat `instance="http://shop-api-1:5000/health"`.

Quand on arrête `shop-api-2` : `up{job="shop-api", instance="shop-api-2:5000"}` passe à 0 (la cible est injoignable) mais `up{job="blackbox-http", instance="http://shop-api-2:5000/health"}` reste à 1 (l'exporter, lui, répond très bien) et c'est `probe_success` qui passe à 0. Avec le blackbox, `up` ne veut pas dire ce qu'on croit ; je m'assure que tout le monde l'a vu.

Division par 86400 pour les jours de certificat.

Partie 4 — Un batch et la Pushgateway 10 min

Énoncé.

1. Lancez le batch : `./lab.sh batch` (il simule une sauvegarde de quelques secondes et pousse trois métriques). Regardez `http://localhost:9091`.
2. Ajoutez le job `pushgateway` avec `honor_labels: true`. Validez, rechargez.
3. Requêtes : `backup_duration_seconds`, `backup_size_megabytes`, et l'ancienneté de la dernière sauvegarde : `time() - backup_last_success_timestamp_seconds`.
4. Sans `honor_labels`, que se passerait-il ? Essayez.

Corrigé.

```
- job_name: pushgateway
  honor_labels: true
  static_configs:
    - targets: ["pushgateway:9091"]
```

Le script (`scripts/batch-job.sh`) pousse sur `/metrics/job/backup/instance/nightly` : les labels `job="backup"` et `instance="nightly"` font partie de la donnée. Sans `honor_labels`, Prometheus les écraserait avec `job="pushgateway"` et `instance="pushgateway:9091"` (et renommerait les originaux `exported_job`, `exported_instance`). Avec, il les respecte.

Je relance le batch deux fois : les valeurs changent, mais la Pushgateway ne garde que la dernière. Et si le batch ne tourne plus jamais, la métrique reste là, figée, avec son timestamp qui vieillit : c'est exactement ce qu'on veut pour alerter.

Partie 5 — Chasse à la cardinalité (bonus, 10 min)

Énoncé.

1. Quelles sont les dix métriques qui ont le plus de séries ? `topk(10, count by (__name__))` (`{__name__=~".+"}`).
2. Combien de séries au total ? `prometheus_tsdb_head_series`. Combien ajoutées par le job `redis` ? `count({job="redis"})`.
3. Discussion : si la boutique passait à 200 routes et 50 instances, combien de séries pour `http_request_duration_seconds_bucket` ? Est-ce raisonnable ?

Corrigé. Les buckets d'histogrammes dominent toujours. 200 routes × 50 instances × 12 buckets × quelques méthodes = plusieurs centaines de milliers de séries pour une seule métrique. Réponse : on réduit les buckets (6 ou 7 bien choisis), on agrège avec des recording rules (jour 2), ou on passe aux native histograms.

17h15 — Récap et quiz du jour 1 15 min

Je fais le tour à l'oral, réponses au tableau :

1. Pull ou push : Prometheus fait quoi, et pourquoi ? (*pull ; résilience, `up`, débogage au `curl`*)
2. Que vaut un Counter brut ? (*rien ; toujours `rate()` ou `increase()`*)

3. Trois labels qu'on ne met jamais sur une métrique. (*user id, request id, IP, email, URL brute...*)
4. Que fait `honor_labels: true` ? (*garde les labels `job / instance` de la cible au lieu de les écraser*)
5. Comment vérifier une configuration avant de recharger ? (*`promtool check config`*)
6. Pourquoi `up` reste à 1 quand une cible du blackbox tombe ? (*on scrape l'exporter, pas la cible ; c'est `probe_success` qui compte*)
7. Le Node Exporter dans Docker Desktop mesure quoi ? (*la VM Linux*)
8. Quel est le suffixe d'un counter ? D'une durée ? (*`_total` , `_seconds`*)

État attendu du `prometheus.yml` ce soir : `jobs` `prometheus` , `shop-api` , `node` , `file-sd` , `redis` , `pushgateway` , `blackbox-http` (le job `node-light` du bonus a été retiré). Le corrigé complet est `solutions/jour-1/prometheus.yml` . Je demande à chacun de le comparer avec le sien avant de partir : demain matin, tout le monde repart du même point.

Je rappelle : ne pas faire `./lab.sh reset` ce soir, on veut de l'historique pour demain. Sur Codespaces, le Codespace peut s'arrêter, les données restent.

Jour 2 — Comprendre : PromQL et Grafana

Objectif de la journée : le matin, PromQL devient un réflexe (rate, agrégations, quantiles, jointures, recording rules). L'après-midi, deux tableaux de bord complets, paramétrables, versionnés dans Git, et une gestion des droits propre.

Heure	Séquence	Durée
9h00	Rappel du jour 1, quiz flash, état des lieux des stacks	15 min
9h15	Module 7 — PromQL, les fondations	40 min
9h55	Exercices série A (2.1 à 2.10)	40 min
10h35	Pause	15 min
10h50	Module 8 — PromQL avancé	40 min
11h30	Exercices série B (2.11 à 2.22)	40 min
12h10	TP 3 — Recording rules et tests	20 min (déborde souvent sur 14h)
12h30	Déjeuner	
14h00	Module 9 — Grafana	35 min
14h35	TP 4 — Dashboard Serveur Linux	60 min
15h35	Pause	15 min
15h50	TP 5 — Dashboard Boutique + dashboards as code	60 min
16h50	Module 10 — Provisioning, utilisateurs, droits, OSS vs Enterprise + exercices 2.30 à 2.33	30 min
17h20	Récap, quiz	10 min

9h00 — Rappel et état des lieux 15 min

Quiz flash à l'oral (cinq questions du récap d'hier). Puis chacun lance `./lab.sh status` et vérifie dans Target health que les sept jobs sont UP. Ceux qui ont un trou copient `solutions/jour-1/prometheus.yml` ... que je ne distribue pas : je projette le corrigé et ils complètent. Cinq minutes maximum, on a du PromQL à faire.

Je vérifie que chacun a au moins une heure d'historique ; sinon, rien de grave, les fenêtres [5m] fonctionneront quand même.

Module 7 — PromQL, les fondations 40 min

Objectif. Sélecteurs, types de résultats, opérateurs, agrégations. À la fin, un stagiaire sait lire n'importe quelle requête simple et en écrire une.

Ce que je dis

PromQL n'est pas SQL. Pas de `SELECT`, pas de `FROM`. On nomme une métrique, on filtre par labels, on applique des fonctions et des agrégations. Ça se lit de l'intérieur vers l'extérieur.

Quatre types de résultats.

- *Instant vector* : une valeur par série, à un instant. `up`, `http_requests_total{job="shop-api"}`. C'est ce que Grafana dessine.
- *Range vector* : pour chaque série, toutes les valeurs sur une fenêtre. `http_requests_total[5m]`. Ne se dessine pas ; sert d'entrée aux fonctions comme `rate()`.
- *Scalar* : un nombre. `42`, `time()`, `scalar(...)`.
- *String* : rarissime.

Les sélecteurs.

Matcher	Sens	Exemple
<code>=</code>	égal	<code>{job="shop-api"}</code>
<code>!=</code>	différent	<code>{route!="metrics"}</code>
<code>=~</code>	regex (ancrée : <code>^...\$</code>)	<code>{status=~"5.."}</code>
<code>!~</code>	regex négative	<code>{device!~"lo.*"}</code>

Les regex sont ancrées : `status=~"5.."` matche exactement trois caractères. Pour « commence par », `=~"5.*"`. Depuis Prometheus 3, le `.` matche aussi les sauts de ligne (rare en pratique).

Le nom de la métrique est lui-même un label, `__name__` : `{__name__=~"shop_.*"}` renvoie toutes les métriques métier. Pratique pour la chasse à la cardinalité.

Décalages dans le temps. `shop_cart_items offset 1h` : la valeur il y a une heure.

`http_requests_total @ 1790400000` : la valeur à un timestamp précis. `@ end()` et `@ start()` dans Grafana pour figer sur la plage affichée. Utile pour comparer « aujourd'hui vs hier » sur un même graphique : `rate(x[5m])` et `rate(x[5m] offset 1d)`.

Les opérateurs. Arithmétiques (`+` `-` `*` `/` `%` `^`), de comparaison (`==` `!=` `>` `<` `>=` `<=`) et logiques (`and` `or` `unless`). Entre deux vecteurs, Prometheus apparie les séries dont **tous les labels sont identiques**.

`node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes` marche parce que les deux ont exactement `{instance, job}`. Sinon, c'est le module 8 (on, `ignoring`).

Un opérateur de comparaison **filtre** : `shop_stock_units < 20` ne renvoie que les produits sous le seuil, avec leur valeur. Avec `bool (shop_stock_units < bool 20)`, il renvoie 0 ou 1 pour toutes les séries. C'est la base des alertes : une alerte se déclenche quand la requête renvoie **au moins une série**, pas quand une valeur est « vraie ».

Les agrégations. `sum`, `min`, `max`, `avg`, `count`, `topk(k, ...)`, `bottomk`, `count_values`, `stddev`, `quantile(0.9, ...)`, `group`.

Deux modificateurs :

- `by (labels)` : garde ces labels, agrège le reste. `sum by (route) (...)` : une série par route.
- `without (labels)` : jette ces labels, garde tous les autres. Plus robuste quand on ne sait pas quels labels existent.

`sum(rate(http_requests_total[5m]))` sans `by` : un seul nombre, tous labels confondus.

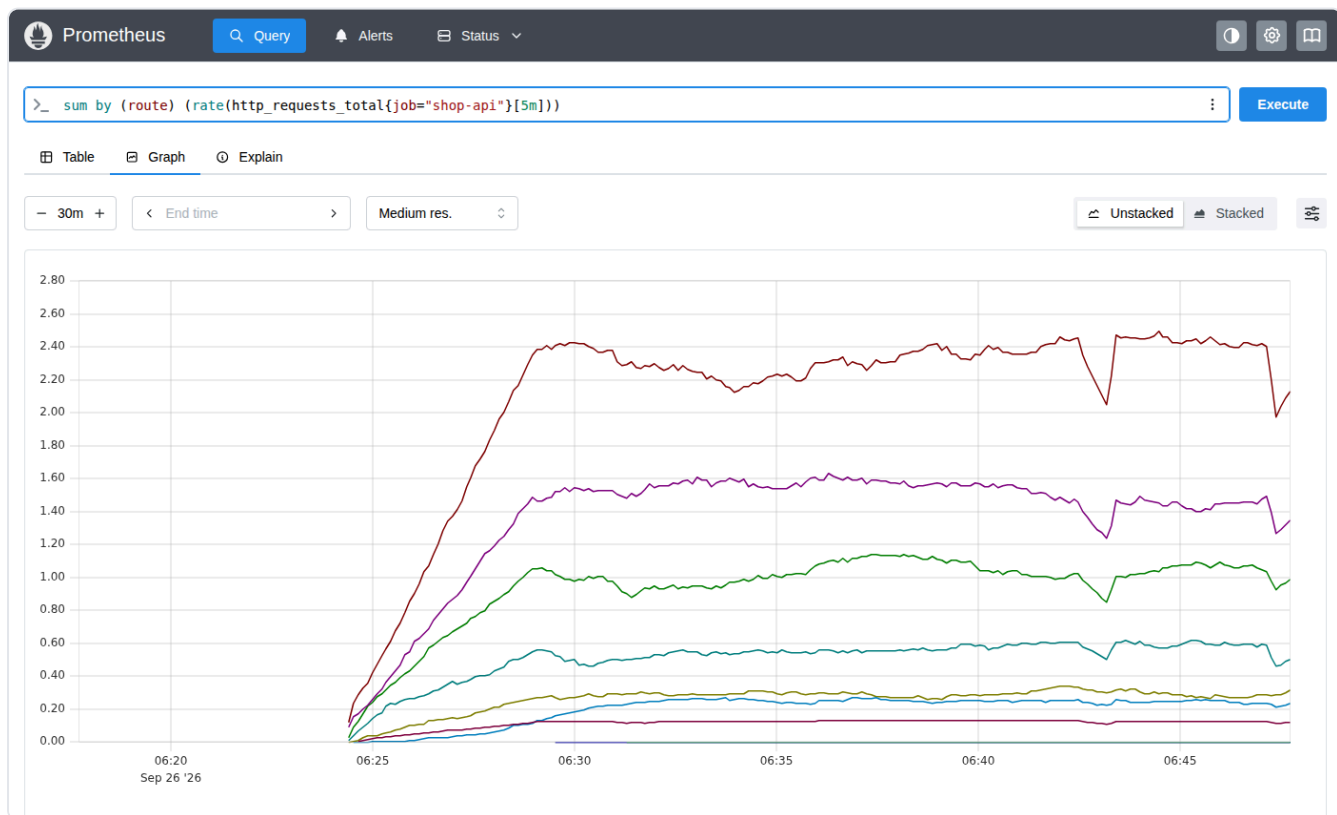
Les fonctions à connaître aujourd'hui. `rate`, `increase`, `irate` (module 8), `histogram_quantile` (module 8), `time()`, `abs`, `round`, `clamp_min/max`, `sort`, `sort_desc`, `label_replace`, `absent`, `changes`, `*_over_time`.

Analogie pour `by` et `without` : un tableur avec une colonne par label. `sum by (route)`, c'est un tableau croisé dynamique avec `route` en ligne et rien en colonne.

Ce que je montre

Dans Prometheus, onglet Query, chaque concept avec sa requête, en montrant le résultat en Table puis en Graph. L'onglet *Explain* de Prometheus 3 décompose l'arbre de la requête : je l'utilise sur `sum by (route) (rate(http_requests_total{job="shop-api"}[5m]))`.

Je montre aussi la différence Table/Graph : Table = instant vector à l'instant T, Graph = la même requête évaluée à chaque pas de temps (c'est ce que fait Grafana avec `query_range`).



L'onglet Graph de Prometheus 3 : débit par route

Exercices série A — Sélection et agrégation 40 min

Format : les stagiaires travaillent seuls dans l'interface Prometheus (pas Grafana, pour se concentrer sur le langage). Je passe dans les rangs. Toutes les cinq minutes environ, je projette le corrigé d'un ou deux exercices.

Énoncés et corrigés.

2.1 — Toutes les cibles et leur état. `up` — 11 séries (tous les jobs du jour 1, dont quatre sondes blackbox), valeur 1 partout si tout va bien. `up{job="shop-api"} : 2`.

2.2 — Les requêtes HTTP en erreur (4xx ou 5xx) sur la boutique, sans le `/metrics`.
`http_requests_total{job="shop-api", status=~"4..|5..", route!="metrics"} Erreur classique : status=~"4|5.." (la regex est ancrée, 4 seul ne matche pas 404).`

2.3 — Les cinq dernières minutes de `http_requests_total` pour shop-api-1 sur `/api/checkout` (range vector). Combien de points ? Pourquoi ? `http_requests_total{instance="shop-api-1:5000", route="/api/checkout"}[5m]` — environ 20 points par série (300 s / 15 s de scrape). Le graphique refuse de l'afficher : un range vector n'est pas dessinaable.

2.4 — Pourcentage de mémoire disponible sur le serveur. `100 * node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes`. Appariement automatique sur `{instance, job}`.

2.5 — Combien d'articles y avait-il dans les paniers il y a 10 minutes ? Et l'écart avec maintenant ?
`shop_cart_items offset 10m` puis `shop_cart_items - shop_cart_items offset 10m`.

2.6 — Nombre total de requêtes reçues par instance, tous codes et routes confondus. `sum by (instance) (http_requests_total{job="shop-api"})` Question piège : est-ce que ça veut dire quelque chose ? Un peu : c'est le total depuis le démarrage. Ce n'est pas un débit.

2.7 — Même chose, mais en gardant tout sauf `method`, `status`, `route`. `sum without (method, status, route) (http_requests_total{job="shop-api"})` Résultat identique à 2.6 si `env` et `team` sont constants ; la différence, c'est que `without` garde `env` et `team` dans le résultat. On compare les deux sorties.

2.8 — Les trois produits les plus en stock, et le moins en stock. `topk(3, shop_stock_units)` (attention : deux instances, donc chaque produit apparaît deux fois ; d'où `topk(3, max by (product) (shop_stock_units))`) et `bottomk(1, ...)`.

2.9 — Les produits dont le stock est sous 60 unités. Puis la même chose en 0/1 pour tous les produits.
`shop_stock_units < 60` puis `shop_stock_units < bool 60`.

2.10 — Combien de routes distinctes la boutique a-t-elle servies ? (Deux agrégations imbriquées.)
`count(count by (route) (http_requests_total{job="shop-api"}))` Le `count by (route)` intérieur donne une série par route ; le `count` extérieur les compte. Pattern à retenir : « combien de valeurs distinctes pour ce label ».

Ce que je vérifie. Que personne n'est bloqué sur la syntaxe des accolades. Les stagiaires les plus rapides font 2.10 en 3 minutes ; je leur demande d'écrire la même chose pour « combien de codes HTTP distincts par route » (`count by (route) (count by (route, status) (...))`).

Module 8 — PromQL avancé 40 min

Objectif. `rate` et ses cousins, les histogrammes, les sous-requêtes, les jointures, les recording rules, et ce qui fait qu'une requête est lente.

Ce que je dis

`rate`, `irate`, `increase`. Les trois travaillent sur un counter et un range vector.

- `rate(x[5m])` : pente moyenne par seconde sur 5 min. Lisse, robuste, la fonction pour les graphiques et les alertes.
- `irate(x[5m])` : pente entre les deux derniers points seulement. Réactif, nerveux, à réserver aux graphiques à haute résolution. Jamais dans une alerte.
- `increase(x[1h])` : combien le compteur a augmenté sur la fenêtre. C'est `rate × durée`. Peut donner des décimales (extrapolation) : « 5,53 commandes » n'a pas de sens mais est mathématiquement correct. `round()` si besoin.

Toutes les trois gèrent les remises à zéro (redémarrage) : un counter qui repasse à 0 n'est pas compté comme une chute.

Règle de la fenêtre : au moins **4 × scrape_interval**, donc `[1m]` minimum avec un scrape à 15 s ; `[5m]` en pratique. Trop court, des trous ; trop long, on lisse les pics. Dans Grafana, `$__rate_interval` calcule la bonne fenêtre selon le zoom : on l'utilisera partout.

Le taux d'erreur. Le ratio qu'on retrouve dans tous les dashboards :

```
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))
```

Piège : si aucune 5xx n'a jamais été vue, le numérateur est **vide** (pas zéro) et le résultat est vide. On y remédie avec `or vector(0)` sur le numérateur, ou on accepte que « pas de données » veuille dire « pas d'erreur ». Je fais réfléchir sur les conséquences pour une alerte.

Les histogrammes. `http_request_duration_seconds_bucket{le="0.5"}` compte les requêtes de moins de 500 ms (cumulatif). Le p95 :

```
histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
```

Décomposition : `rate` sur chaque bucket (ce sont des counters) ; `sum by (le)` agrège les instances **en gardant 1e**, sans quoi la fonction ne peut plus rien faire ; la fonction interpole linéairement dans le bucket où tombe le 95e centile. On ajoute `route` dans le `by` pour un p95 par route.

Le p95 est borné par la précision des buckets. Je le montre : avec nos buckets, un p99 de la route checkout sous chaos affiche des valeurs suspicieusement rondes (2,5 s, 5 s).

Latence moyenne : `rate(x_sum[5m]) / rate(x_count[5m])`. Toujours vraie, jamais utile pour un SLO (la moyenne cache les pires cas).

Native histograms. Un seul échantillon par série au lieu d'un par bucket, buckets exponentiels automatiques, précision bien meilleure. `histogram_quantile(0.95, sum(rate(x[5m])))` sans `by (le)`. Stables depuis Prometheus 3.9, il suffit de `scrape_native_histograms: true` dans la configuration... et

d'une bibliothèque cliente qui les produit, ce qui reste le point bloquant en 2026. On les regarde de loin ; ils deviendront la norme.

Les fonctions `_over_time` . Elles s'appliquent aux gauges sur un range vector :

`max_over_time(shop_cart_items[1h])`, `avg_over_time`, `min_over_time`, `last_over_time`, `quantile_over_time(0.9, ...)`, `count_over_time`, `changes` (nombre de changements de valeur), `resets`, `deriv` (pente d'une gauge), `predict_linear(x[6h], 24*3600)` (extrapolation : le disque sera-t-il plein demain ?), `delta` .

Les sous-requêtes. `rate(x[1m])[1h:1m]` : évalue `rate(x[1m])` toutes les minutes sur une heure et renvoie ça comme range vector. On peut alors faire `max_over_time(rate(x[1m])[1h:1m])` : le pic de débit sur l'heure. Coûteux, à réserver aux besoins précis.

Les jointures (vector matching). Quand les labels ne sont pas identiques des deux côtés :

- `on (labels)` : n'apparier que sur ces labels. `ignoring (labels)` : apparier sur tout sauf ceux-là.
- `group_left` / `group_right` : quand un côté a plusieurs séries pour une seule de l'autre (many-to-one). `group_left(version)` copie en plus le label `version` du côté droit.

L'exemple canonique avec une info metric :

```
shop_stock_units * on (instance) group_left(version) shop_app_info
```

Chaque série de stock récupère le label `version` de son instance. On peut ensuite faire `sum by (version) (...)` : « combien de stock géré par la version 1.4.2 ? ». Même technique en Kubernetes avec `kube_pod_labels` .

`label_replace(up, "pod", "$1", "instance", "(.*) : 5000")` : crée un label `pod` à partir de `instance` par regex. Utile pour faire correspondre des labels de sources différentes.

`absent()` . `absent(up{job="shop-api"})` renvoie 1 si la série n'existe pas du tout. C'est la seule façon d'alerter sur « cette métrique a disparu » (une cible retirée de la configuration par erreur, une application qui n'expose plus une métrique).

Les recording rules. Une requête coûteuse ou réutilisée partout se pré-calcule : Prometheus l'évalue toutes les 15 s et stocke le résultat comme une nouvelle série. Convention de nommage

`niveau:metrique:operations : job:http_requests:rate5m`, `instance:node_cpu_utilisation:ratio_rate5m` . Avantages : dashboards instantanés, alertes simples, et l'agrégation qu'on envoie au stockage longue durée est déjà faite.

Ce qui rend une requête lente. Le nombre de séries touchées × le nombre de points par série.

`rate(http_requests_total[5m])` sans filtre sur un an de données, c'est des milliards de points.

Réflexes :

- filtrer par `job` ou nom exact avant d'agréger ;
- éviter `{__name__=~"..."}` et les regex larges ;
- fenêtres courtes, `_over_time` et sous-requêtes avec parcimonie ;
- recording rules pour ce qui est affiché en permanence ;
- regarder `prometheus_engine_query_duration_seconds` et le *Query inspector* de Grafana.

Prometheus 3 a un onglet *Explain* et l'API `/api/v1/query?stats=all` qui donne le nombre d'échantillons lus. Je montre.

Ce que je montre

- La courbe de `rate` contre `irate` sur `shop_orders_total` : je lance `./lab.sh traffic 30` puis reviens à 6, et on voit `irate` réagir en un point et `rate` en cinq minutes.
- `histogram_quantile` avec et sans `by (le)` : sans, résultat vide ou NaN.
- La jointure `shop_stock_units * on (instance) group_left(version) shop_app_info` en Table, pour voir le label `version` arriver.

Exercices série B — Taux, quantiles, jointures 40 min

2.11 — Débit de requêtes par seconde, par route, sur la boutique. `sum by (route)`

`(rate(http_requests_total{job="shop-api"}[5m]))` — `/api/products` domine.

2.12 — Combien de commandes ont été passées dans la dernière heure ? Par moyen de paiement ?

`sum(increase(shop_orders_total[1h]))` puis `sum by (payment_method)`
`(increase(shop_orders_total[1h]))` . Je fais remarquer les décimales et je fais ajouter `round()` .

2.13 — Chiffre d'affaires par heure (en euros/h) à partir de `shop_revenue_euros_total` .

`sum(rate(shop_revenue_euros_total[5m])) * 3600` . C'est le panneau vedette de cet après-midi.

2.14 — Taux d'erreur 5xx global, en pourcentage. Puis lancez `./lab.sh chaos errors on` , attendez deux minutes, observez, puis `./lab.sh chaos errors off` . `100 *`

`sum(rate(http_requests_total{job="shop-api", status=~"5.."}[5m])) /`
`sum(rate(http_requests_total{job="shop-api"}[5m]))` Il monte vers 35-40 % puis redescend lentement (la fenêtre de 5 min lisse).

2.15 — Latence p50, p95 et p99 sur toutes les routes. Puis uniquement sur `/api/checkout` .

`histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket{job="shop-api"}[5m])))` , et avec `route="/api/checkout"` dans le sélecteur. Les trois quantiles dans le même graphique en ajoutant trois requêtes (dans Grafana tout à l'heure).

2.16 — Latence moyenne sur la même métrique. Comparez-la au p95.

`sum(rate(http_request_duration_seconds_sum{job="shop-api"}[5m])) /`
`sum(rate(http_request_duration_seconds_count{job="shop-api"}[5m]))` La moyenne est très en dessous du p95 : la distribution a une queue.

2.17 — CPU utilisé en %, par instance, à partir de `node_cpu_seconds_total` . Puis la répartition par mode

(`user` , `system` , `iowait` ...) sans `idle` . `100 * (1 - avg by (instance)`
`(rate(node_cpu_seconds_total{mode="idle"}[5m])))` puis `sum by (mode)`
`(rate(node_cpu_seconds_total{mode!="idle"}[5m]))` .

2.18 — Débit réseau entrant en bits/s, hors interfaces `lo` , `veth*` , `br*` , `docker*` . `sum by (device)`

`(rate(node_network_receive_bytes_total{device!~"lo|veth.*|br.*|docker.*"}[5m])) * 8`

2.19 — Le pic du nombre d'articles dans les paniers sur la dernière heure, et le pic de débit de requêtes sur la dernière heure (sous-requête). `max_over_time(shop_cart_items[1h])` puis

`max_over_time(sum(rate(http_requests_total{job="shop-api"}[1m]))[1h:1m])` .

2.20 — Ajoutez le label `version` (de `shop_app_info`) aux séries de débit par instance. `sum by (instance) (rate(http_requests_total{job="shop-api"}[5m])) * on (instance) group_left(version) shop_app_info` Piège : `sum by (instance)` est obligatoire à gauche, sinon il y a plusieurs séries par

instance des deux côtés et Prometheus refuse (many-to-many).

2.21 — Prometheus scrape-t-il un job `paielement` ? Écrivez la requête qui renverrait 1 s'il n'existe pas.
`absent(up{job="paielement"})` → 1. `absent(up{job="shop-api"})` → vide.

2.22 — À ce rythme, combien vaudra `shop_revenue_euros_total` dans une heure ? Et dans combien de temps le disque / sera-t-il plein ? (`predict_linear`)

```
predict_linear(shop_revenue_euros_total[10m], 3600) ;
predict_linear(node_filesystem_avail_bytes{mountpoint="/"}[1h], 24*3600) : négatif = plein avant 24 h. Sur nos machines de lab, le disque bouge peu, la prédiction est bruitée ; c'est normal, c'est un signal à moyen terme.
```

Ce que je vérifie. Le `by (1e)` dans 2.15 : c'est l'erreur numéro un. Et que le chaos a bien été remis à `off` .

TP 3 — Recording rules et tests unitaires 20 min + débordement

Objectif. Pré-calculer ce qui sera affiché en permanence, valider les règles, et découvrir que des règles PromQL se testent comme du code.

Mise en situation. Les panneaux « débit », « taux d'erreur », « p95 » et « CA par heure » vont être affichés sur un écran mural 24 h/24. On ne veut pas que Prometheus recalcule les histogrammes toutes les cinq secondes.

Partie 1 — Écrire les règles 10 min

Énoncé. Dans `prometheus/rules/recording.yml`, groupe `shop_api_recording`, créez :

1. `instance_route:http_requests:rate5m` — débit par instance et route.
2. `job:http_requests:rate5m` — débit total du job.
3. `instance:http_errors:ratio_rate5m` — ratio d'erreurs 5xx par instance (0 à 1).
4. `route:http_request_duration_seconds:p95_5m` — p95 par route.
5. `job:shop_revenue_euros:rate1h` — CA en euros par heure.

Puis, dans un groupe `node_recording` (intervalle 30 s) :

6. `instance:node_cpu_utilisation:ratio_rate5m` — CPU utilisé (0 à 1).
7. `instance:node_memory_utilisation:ratio` — mémoire utilisée (0 à 1).

Validez (`./lab.sh check`), rechargez, vérifiez dans **Status → Rules** et interrogez `job:http_requests:rate5m` .

Corrigé. Le fichier complet est `solutions/jour-2/recording.yml` . Les points à commenter :

- `interval: 15s` au niveau du groupe surcharge `evaluation_interval` .
- Le ratio d'erreur garde `by (instance)` des deux côtés, sinon appariement impossible.
- Le p95 garde `route` et `le` dans le `sum by`, et se limite aux routes `/api/.*` : sans trafic, un quantile vaut NaN et polluerait les tables.
- On stocke des ratios (0-1), pas des pourcentages : Grafana convertira (`percentunit`), et les alertes compareront à `0.05`, pas à `5` . Une convention, à tenir partout.

Partie 2 — Tester les règles 10 min

Énoncé. Ouvrez `prometheus/tests/alerts_test.yml` : il teste l'alerte `TargetDown` avec des séries simulées. Lancez `./lab.sh test`. Puis ajoutez un test pour votre recording rule

`job:http_requests:rate5m` : avec une série `http_requests_total{job="shop-api", instance="a"}` qui vaut `0+10x40` (0, 10, 20... toutes les 15 s), que doit valoir la règle à `eval_time: 5m` ?

Corrigé. À 15 s d'intervalle, +10 par scrape = $10/15 \approx 0,667$ req/s.

```
rule_files:
- ../rules/recording.yml
evaluation_interval: 15s
tests:
- interval: 15s
  input_series:
  - series: 'http_requests_total{job="shop-api", instance="a", route="/", method="GET", status="200"}'
    values: "0+10x40"
  promql_expr_test:
  - expr: job:http_requests:rate5m
    eval_time: 5m
    exp_samples:
    - labels: 'job:http_requests:rate5m{job="shop-api"}'
      value: 0.6666666666666666
```

`promtool test rules` est peu connu et pourtant c'est ce qui permet de mettre les règles dans une CI. J'insiste : une règle d'alerte non testée est une règle qui sonnera un dimanche pour rien, ou qui ne sonnera pas le jour où il faut.

Partie 3 — Mesurer le gain (bonus)

Dans Grafana, Explore, comparer avec le Query inspector (onglet *Stats*) le temps d'exécution de `histogram_quantile(0.95, sum by (route, le) (rate(http_request_duration_seconds_bucket[5m])))` et de `route:http_request_duration_seconds:p95_5m`. Sur notre petit lab, la différence est de quelques millisecondes ; en production avec des milliers de séries, c'est des secondes.

Module 9 — Grafana 35 min

Objectif. Comprendre comment Grafana est construit, connaître l'anatomie d'un dashboard et les bonnes pratiques de lisibilité, savoir choisir une visualisation.

Ce que je dis

Sous le capot. Un serveur Go, une base SQLite par défaut (`grafana.db`, ou PostgreSQL / MySQL en production), une interface React. Il ne stocke que sa configuration : utilisateurs, sources de données, dashboards (en JSON), règles d'alerte. Pas une seule métrique. Tout ce qu'il affiche vient d'une *source de données* interrogée à la volée.

Configuration par `grafana.ini` ou, dans un conteneur, par variables d'environnement `GF_<SECTION>_<CLÉ>`. Notre compose en utilise quelques-unes (mot de passe admin, pas d'inscription, pas de télémétrie).

Les quatre concepts.

1. *Data source* : une connexion (Prometheus, Loki, PostgreSQL, ...). La nôtre est provisionnée par fichier YAML : personne ne l'a cliquée.
2. *Panel* : une requête + une visualisation + des options. Un panel répond à une question.
3. *Dashboard* : des panels, des variables, des annotations, des liens. Un dashboard répond à un besoin (« la boutique va-t-elle bien ? »), pas à tous les besoins.
4. *Folder* : pour ranger et pour donner des droits.

Grafana 13. Ce qui a changé et qui compte pour nous : les *dynamic dashboards* sont la norme (nouvelle barre latérale d'édition, deux modes de disposition : *Custom grid* où on place chaque panel, *Auto grid* où ils se rangent seuls ; regroupements en *rows* et *tabs* ; règles d'affichage conditionnel), l'éditeur de variables a été refait, on peut restaurer un dashboard supprimé, *Git Sync* est disponible en OSS pour versionner les dashboards directement dans un dépôt. Le plugin de rendu d'images (`grafana-image-renderer`) a été retiré : plus d'export PNG côté serveur.

Explore avant de construire. *Explore* (menu de gauche) est un bac à sable : on tape une requête, on regarde, on ajuste, puis « Add to dashboard ». On ne construit jamais un panel à l'aveugle.

Choisir la visualisation. Un choix par question :

La question	La visualisation
Ça évolue comment ?	Time series
Ça vaut combien, là, maintenant ?	Stat (avec une sparkline)
C'est à quel niveau sur une échelle bornée ?	Gauge
Comparer quelques valeurs entre elles	Bar gauge (horizontal)
Une liste avec plusieurs colonnes	Table
Une répartition	Pie chart (avec parcimonie : 5 parts maximum)
Un état dans le temps (up/down, ok/warn)	State timeline, Status history
Une distribution qui évolue (latences)	Heatmap
Un texte, une consigne, un lien	Text
Les alertes en cours	Alert list

Ce qui rend un dashboard lisible. Mes règles, dans l'ordre d'importance :

1. **Les unités.** Jamais un « 38491023 » brut. `bytes` → « 36,7 MiB », `seconds` → « 1,2 s », `percentunit` pour un ratio 0-1, `reqps` pour un débit, `currencyEUR`. Grafana convertit tout.
2. **Les seuils et les couleurs.** Vert/orange/rouge cohérents sur tout le dashboard. Un seuil se voit sur un Stat, un Gauge, une Table (fond coloré) et même sur un Time series (ligne pointillée).
3. **Les value mappings.** `1` → « UP » en vert, `0` → « DOWN » en rouge. Un dashboard doit être lisible par quelqu'un qui ne connaît pas Prometheus.
4. **La légende.** `{{route}}` au lieu de `{route="/api/cart", instance="..."}`. En mode table avec `Mean` et `Max` quand il y a plus de trois séries.

5. **L'ordre de lecture.** En haut à gauche : les indicateurs qui répondent à « ça va ? ». En dessous, le détail. En bas, les données brutes. Les rows regroupent.

6. **Pas plus de 10-12 panels visibles.** Au-delà, on crée un second dashboard et un lien.

Variables et templating. Un menu déroulant en haut qui filtre tous les panels. Types : *Query* (`label_values(node_uname_info, instance)`), *Custom* (liste fixe), *Textbox*, *Interval*, *Data source*, *Constant*, et le nouveau *Switch* (un interrupteur on/off). Dans les requêtes : `$instance` ou `${instance}`. Multi-valeur + « All » → il faut `=~` et non `=` (Grafana génère `a|b|c`). Les variables peuvent se chaîner : `$route` dépend de `$instance`.

Les variables intégrées : `$__rate_interval` (la bonne fenêtre pour `rate`), `$__interval`, `$__range` (la plage affichée, pour `increase(x[$__range])`), `$__from` / `$__to`.

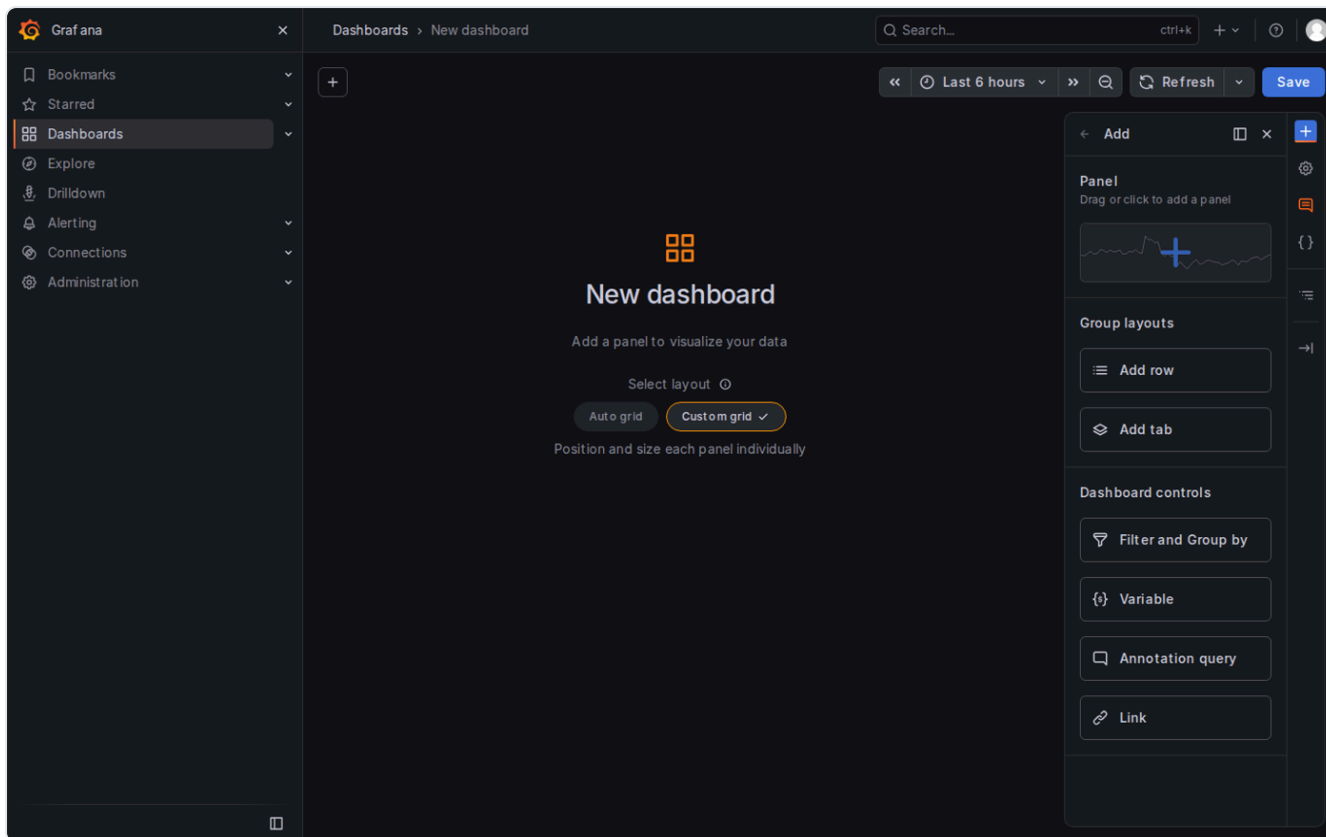
Transformations. Elles travaillent sur le résultat, côté Grafana : fusionner deux requêtes (*Merge*), renommer et masquer des colonnes (*Organize fields*), trier (*Sort by*), filtrer, calculer une colonne (*Add field from calculation*), réduire une série à une valeur (*Reduce*). Indispensables pour les tables.

Annotations, seuils, liens. Une annotation est un événement dessiné en trait vertical sur tous les graphiques : un déploiement, une bascule chaos. Source : une requête Prometheus (`changes(shop_chaos_mode[1m]) > 0`) ou une saisie manuelle (Ctrl+clic sur un graphique). Les liens de dashboard (barre du haut) et les data links (clic sur une série) permettent de naviguer du global vers le détail.

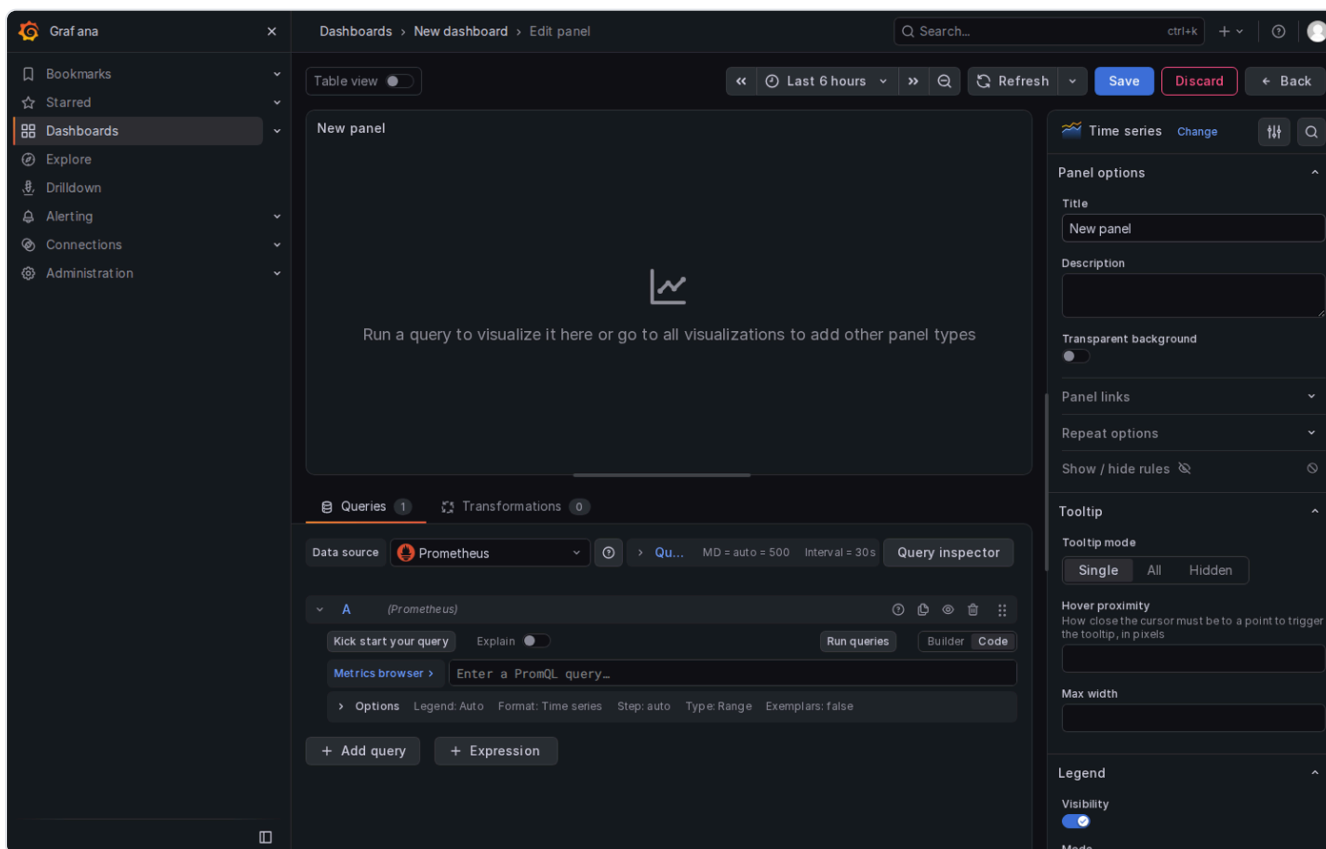
Ce que je montre

Une visite de Grafana 13 en cinq minutes, en projetant :

- **Connections → Data sources** : la source Prometheus, provisionnée, avec le bouton *Explore*.
- **Explore** : `sum(rate(http_requests_total{job="shop-api"}[$__rate_interval]))`, mode *Code*, puis *Builder* pour ceux qui préfèrent cliquer.
- **Dashboards → New → New dashboard** : la barre latérale *Add* (Panel, Add row, Add tab, Variable, Annotation query, Link), le choix *Auto grid* / *Custom grid*.
- Un panel : *Configure visualization*, la requête, *Run queries*, le volet de droite *Suggestions* / *All visualizations*, puis les options (Panel options, Legend, Standard options avec Unit, Thresholds, Value mappings, Add field override).
- *Save*, titre, dossier. *Exit edit*.
- Le dashboard *00 - Bienvenue* provisionné : son cadenas « provisioned ».



Grafana 13 : un nouveau dashboard et sa barre latérale d'édition



Grafana 13 : l'éditeur de panel (requêtes en bas, visualisation et options à droite)

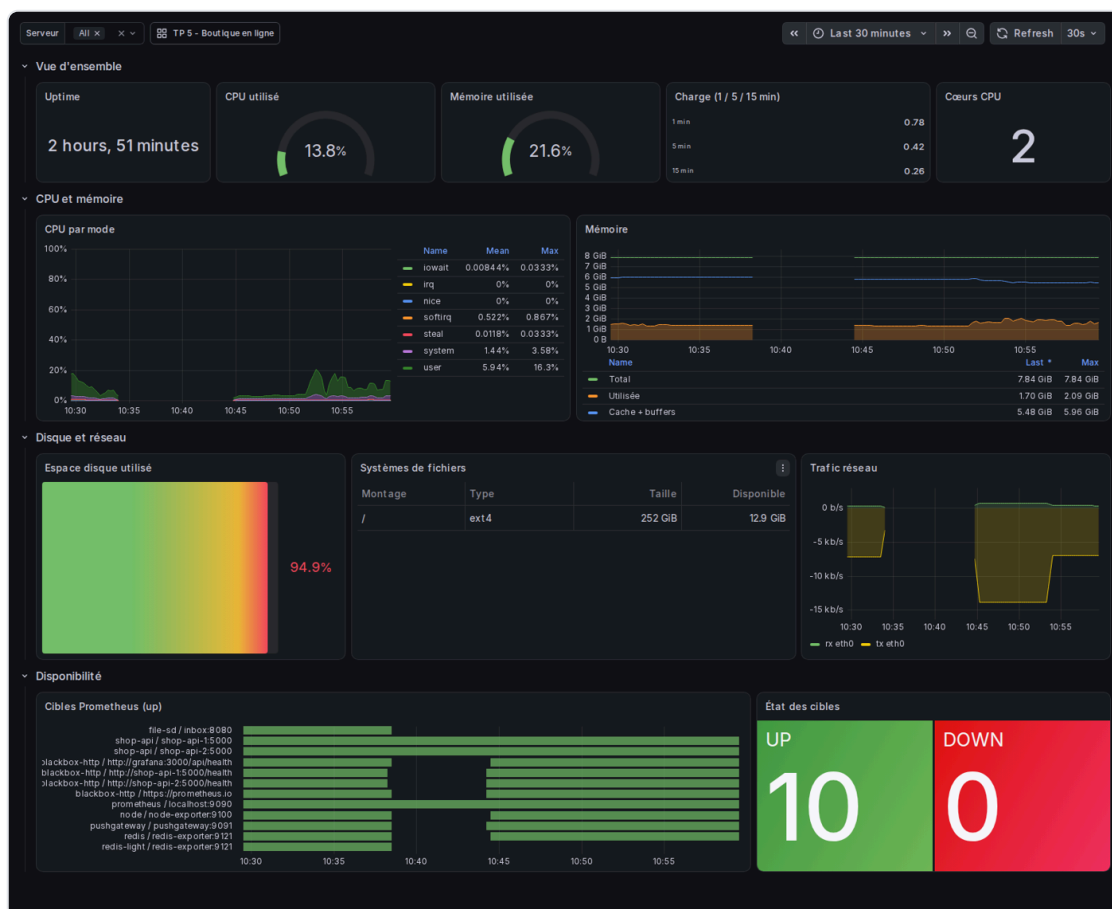
TP 4 — Un tableau de bord paramétrable pour un serveur Linux

60 min

Objectif. Le cas pratique du programme. Dix panels, huit types de visualisation, une variable, des seuils, des unités, des transformations, des rows.

Mise en situation. L'équipe d'exploitation veut un écran par serveur : d'un coup d'œil, on doit savoir s'il va bien, et en dessous avoir le détail CPU, mémoire, disque, réseau. Le même dashboard doit servir pour tous les serveurs : une liste déroulante pour choisir.

Le résultat attendu est projeté (corrigé `solutions/jour-2/dashboards/tp4-serveur-linux.json`, capture dans le guide stagiaire). Les stagiaires ont la liste des panels, les métriques à utiliser et les options attendues, pas les requêtes finales.



Le résultat attendu du TP 4

Étape 0 — Créer le dashboard et la variable 10 min

Énoncé.

1. Dashboards → New → New dashboard. Disposition *Custom grid*.
2. Barre latérale Add → Variable → type Query. Nom `instance`, label `Serveur`, requête `label_values(node_uname_info, instance)`. Cochez *Multi-value* et *Include All value* (valeur personnalisée pour All : `.*`).
3. Sauvegardez sous le nom `TP 4 - Serveur Linux` dans le dossier *Formation*.

Corrigé. Dans Grafana 13, la variable se crée depuis la barre latérale et se modifie en cliquant sur sa puce en haut à gauche (`instance`) → *Open variable editor*. La requête `label_values(...)` est une fonction spéciale de la source Prometheus, pas du PromQL. Avec *All* = `.*`, toutes les requêtes doivent utiliser `instance=~"$instance"`.

Étape 1 — Row « Vue d'ensemble » : cinq indicateurs 15 min

Énoncé. Ajoutez une row « Vue d'ensemble » puis, dedans :

1. **Uptime** — Stat. `time() - node_boot_time_seconds{instance=~"$instance"}`. Unité *Time* → *duration* (*d hh:mm:ss*) ou *dt**durations*. Query type *Instant*.
2. **CPU utilisé** — Gauge. La formule CPU du jour 1 avec `[$__rate_interval]`, unité *Percent (0-100)*, min 0, max 100, seuils vert / orange à 70 / rouge à 90.
3. **Mémoire utilisée** — Gauge. `100 * (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)` avec le filtre *instance*, mêmes seuils.
4. **Charge 1 / 5 / 15 min** — Stat avec trois requêtes (`node_load1`, `node_load5`, `node_load15`), légendes `1 min`, `5 min`, `15 min`, orientation horizontale, 2 décimales.
5. **Cœurs CPU** — Stat. `count(node_cpu_seconds_total{mode="idle", instance=~"$instance"})`.

Corrigé (requêtes exactes).

Panel	Requête
Uptime	<code>time() - node_boot_time_seconds{instance=~"\$instance"}</code>
CPU	<code>100 * (1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle", instance=~"\$instance"}[\$__rate_interval])))</code>
Mémoire	<code>100 * (1 - node_memory_MemAvailable_bytes{instance=~"\$instance"} / node_memory_MemTotal_bytes{instance=~"\$instance"})</code>
Charge	<code>node_load1{instance=~"\$instance"}</code> (+ <code>load5</code> , <code>load15</code>)
Cœurs	<code>count(node_cpu_seconds_total{mode="idle", instance=~"\$instance"})</code>

Chemin des options dans le volet de droite : *Standard options* → *Unit*, *Standard options* → *Min / Max*, *Thresholds* (Add threshold), *Stat styles* → *Orientation*. La légende se règle dans *Options* → *Legend* de la requête (champ *Legend* : *Custom*, valeur `1 min`).

Ce que je fais remarquer : sur la Gauge, sans `avg by (instance)`, on aurait une jauge par cœur. Et si la variable est sur *All* avec deux serveurs, on aurait deux jauges : c'est voulu, c'est le comportement multi-valeur.

Étape 2 — Row « CPU et mémoire » : deux Time series 10 min

Énoncé.

6. **CPU par mode** — Time series empilé (*Graph styles* → *Stack series* : *Normal*), *Fill opacity* 40, unité *Percent (0.0-1.0)*, min 0 max 1. Requête : `rate` par mode hors `idle`, divisé par le nombre de cœurs. Légende `{{mode}}`, mode *Table* à droite avec *Mean* et *Max*.

7. **Mémoire** — Time series, trois requêtes : Total, Utilisée (Total – Disponible), Cache + buffers (`node_memory_Cached_bytes + node_memory_Buffers_bytes`). Unité *bytes (IEC)*. Un *field override* pour colorer « Utilisée » en orange avec un remplissage.

Corrigé.

```
sum by (mode) (rate(node_cpu_seconds_total{mode!="idle", instance=~"$instance"}
[ $__rate_interval ]))
/ scalar(count(node_cpu_seconds_total{mode="idle", instance=~"$instance"}))
```

`scalar()` est nécessaire : `count(...)` renvoie un vecteur sans labels, qui ne s'apparie pas avec des séries portant `mode`. Sans `scalar`, panel vide. C'est le piège de l'étape.

Override : *Add field override* → *Fields with name : Utilisée* → *Add override property : Color scheme* → *Single color (orange)* puis *Fill opacity : 30*.

Étape 3 — Row « Disque et réseau » : Bar gauge, Table, Time series 15 min

Énoncé.

8. **Espace disque utilisé** — Bar gauge horizontal, mode *Gradient*, une barre par `mountpoint`, unité Percent, seuils 75 / 90. Exclure `fstype tmpfs|overlay|squashfs`. Query type *Instant*.

9. **Systèmes de fichiers** — Table. Deux requêtes *Instant* au format *Table* :

`node_filesystem_size_bytes` et `node_filesystem_avail_bytes` (mêmes filtres). Transformations : *Join by field* (champ `mountpoint`, mode *Outer*), puis *Organize fields* pour masquer les colonnes `Time`, `__name__`, `job`, `instance`, `device` (en double, suffixées 1 et 2) et renommer `Value #A` → `Taille`, `Value #B` → `Disponible`, `mountpoint` → `Montage`, `fstype 1` → `Type`, puis *Sort by Disponible* croissant. Unité bytes.

10. **Trafic réseau** — Time series, réception en positif, émission en négatif (multiplier par -1), unité *bits/sec*, légende `rx {{device}}` / `tx {{device}}`.

Corrigé.

Bar gauge : `100 * (1 - node_filesystem_avail_bytes{fstype!~"tmpfs|overlay|squashfs", instance=~"$instance"} / node_filesystem_size_bytes{fstype!~"tmpfs|overlay|squashfs", instance=~"$instance"})`, légende `{{mountpoint}}`.

Table : les deux requêtes en *Format : Table* et *Type : Instant* (menu *Options* sous la requête). Sans transformation, on obtient deux tables. *Join by field* les apparie sur le champ choisi (`mountpoint`) ; *Merge* aurait aussi pu marcher mais il apparie sur *tous* les labels communs et se montre capricieux dès qu'un label diffère. *Organize fields* permet aussi de réordonner par glisser-déposer.

Réseau : `sum by (device) (rate(node_network_receive_bytes_total{device!~"lo|veth.*|br.*|docker.*", instance=~"$instance"} [ $__rate_interval ])) * 8` et `- sum by (device) (rate(node_network_transmit_bytes_total{...} [ $__rate_interval ])) * 8`.

Étape 4 — Row « Disponibilité » : State timeline et Stat coloré 10 min

Énoncé.

11. **Cibles Prometheus** — State timeline sur `up`, légende `{{job}} / {{instance}}`. Value mappings : `1` → UP (vert), `0` → DOWN (rouge). *Merge equal consecutive values* activé.
12. **État des cibles** — Stat, deux requêtes `sum(up)` (UP) et `count(up == 0) or vector(0)` (DOWN), *Color mode : Background*, override couleur verte / rouge.
13. Sauvegardez. Testez la variable : choisissez une instance, puis *All*. Changez la plage de temps. Retrouvez le chaos CPU d'hier (ou relancez-en un).

Corrigé. Le `or vector(0) : count(up == 0)` est vide quand tout va bien, et un Stat vide affiche « No data ». `or vector(0)` fournit un zéro par défaut. Pattern à retenir pour tous les compteurs d'anomalies.

Value mappings : *Value mappings* → *Add value mappings* → *Value : 1, Display text : UP, Color : green*.

Ce que je vérifie. Que le dashboard est sauvegardé dans le dossier *Formation* avec le bon titre (le TP 5 y fera un lien). Que la variable fonctionne avec `=~`. Les rapides ajoutent un panel « Processus » (`node_processes_state`, pie chart) ou passent le dashboard en *Auto grid* pour comparer.

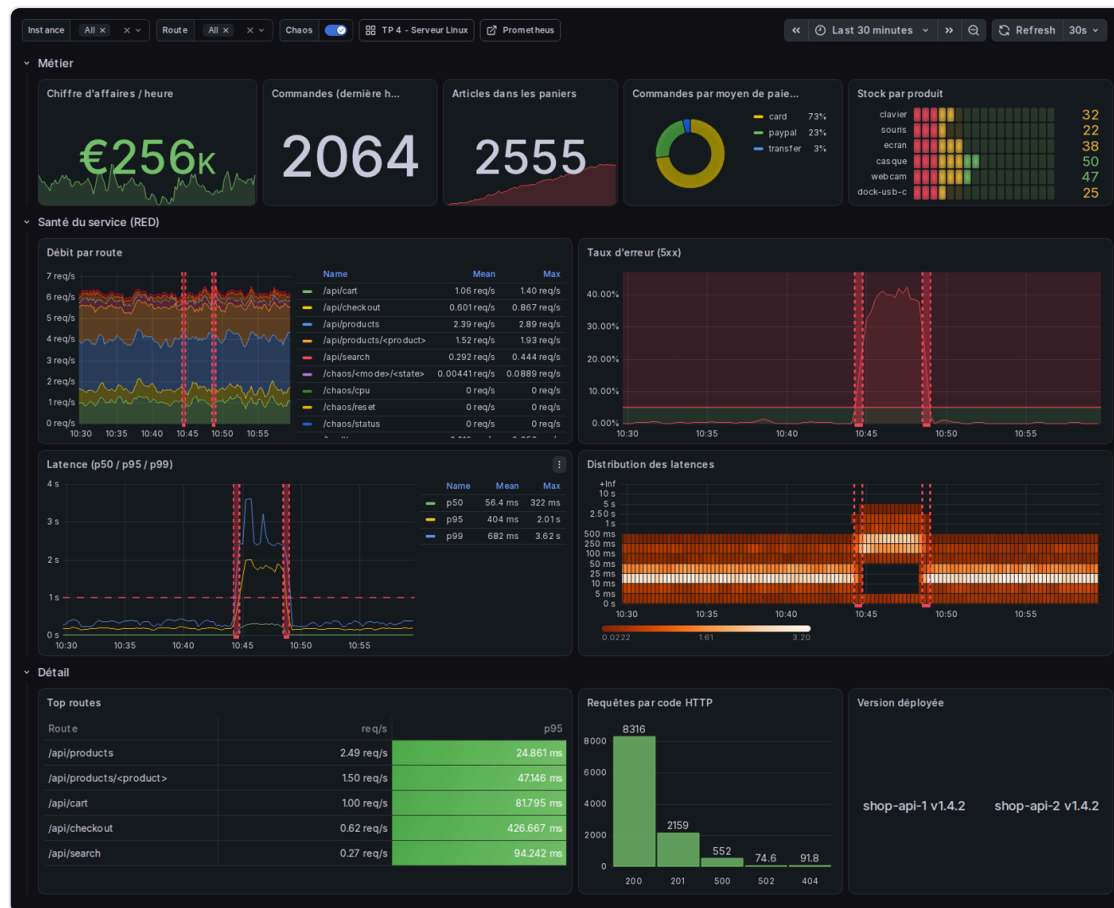
TP 5 — Le tableau de bord de la boutique, puis dashboards as code

60 min

Objectif. Métier + RED, deux variables chaînées, annotations, liens, heatmap, transformations avancées ; puis exporter en JSON et provisionner.

Mise en situation. Le directeur commercial veut un écran avec le chiffre d'affaires, les commandes, les paniers, les stocks. L'équipe technique veut, sur le même écran, le débit, les erreurs et la latence, filtrables par instance et par route. Et tout le monde veut voir sur les graphiques « quand est-ce qu'on a cassé quelque chose ».

Résultat attendu projeté : `solutions/jour-2/dashboards/tp5-boutique.json`.



Le résultat attendu du TP 5 (avec deux bascules chaos annotées)

Étape 0 — Variables chaînées 5 min

Énoncé. Nouveau dashboard TP 5 - Boutique en ligne, dossier Formation. Deux variables Query, multi-valeur avec All :

- `instance` : `label_values(http_requests_total{job="shop-api"}, instance)`
- `route` : `label_values(http_requests_total{job="shop-api", instance=~"$instance"}, route)`

Corrigé. La seconde dépend de la première : changer `$instance` recalcule la liste des routes. Dans l'éditeur de variable, *Refresh* : *On time range change* est le réglage habituel.

Étape 1 — Row « Métier » 15 min

Énoncé. Cinq panels :

- Chiffre d'affaires / heure** — Stat, `rate(shop_revenue_euros_total) * 3600` filtré par `$instance`, unité *Currency* → *Euro* (€), 0 décimale, sparkline (*Graph mode* : *Area*), couleur fixe verte.
- Commandes (dernière heure)** — Stat, `increase(...[1h])` sommé, *Instant*.
- Articles dans les paniers** — Stat avec sparkline, `sum(shop_cart_items{...})`.
- Commandes par moyen de paiement** — Pie chart (donut), `increase(shop_orders_total[$__range])` par `payment_method`, légende à droite avec les pourcentages.
- Stock par produit** — Bar gauge, mode *LCD*, une barre par produit, seuils rouge < 20, orange < 40, vert au-dessus (*Thresholds* : rouge base, orange 20, vert 40), max 120.

Corrigé.

Panel	Requête
CA / h	<code>sum(rate(shop_revenue_euros_total{job="shop-api", instance=~"\$instance"}[\$__rate_interval])) * 3600</code>
Commandes	<code>sum(increase(shop_orders_total{job="shop-api", instance=~"\$instance"}[1h]))</code>
Paniers	<code>sum(shop_cart_items{job="shop-api", instance=~"\$instance"})</code>
Paieement	<code>sum by (payment_method) (increase(shop_orders_total{job="shop-api", instance=~"\$instance"}[\$__range]))</code>
Stock	<code>sum by (product) (shop_stock_units{...}) / count by (product) (shop_stock_units{...})</code>

Le stock : chaque instance a son propre stock (c'est une simulation, en vrai il serait en base) ; on affiche la moyenne des deux. `$__range` dans le pie chart : la répartition s'adapte à la plage affichée. Question à poser : « pourquoi un Stat sur `rate` et pas sur la valeur brute du compteur ? »

Étape 2 — Row « Santé du service (RED) » 15 min

Énoncé. Quatre panels, tous filtrés par `$instance` et `$route` (sauf le taux d'erreur, par instance seulement) :

- Débit par route** — Time series empilé, unité *requests/sec*, légende table à droite (Mean, Max).
- Taux d'erreur (5xx)** — Time series, unité *Percent (0.0-1.0)*, seuil rouge à 0.05 affiché en *Thresholds → Show thresholds : As lines and filled regions*, couleur fixe rouge, sans légende.
- Latence p50 / p95 / p99** — Time series, trois requêtes, unité seconds, seuil pointillé à 1 s (*Show thresholds : As lines (dashed)*).
- Distribution des latences** — Heatmap. Requête `sum by (le) (rate(http_request_duration_seconds_bucket{...}[$__rate_interval]))` en *Format : Heatmap*. Dans les options : *Calculate from data : No*, axe Y en secondes, palette *Oranges*.

Corrigé. Débit : `sum by (route) (rate(http_requests_total{job="shop-api", instance=~"$instance", route=~"$route"}[$__rate_interval]))`. Taux d'erreur : le ratio du 2.14 sans le `100 *` (unité percentunit). p95 : celui du 2.15 avec les filtres. Heatmap : le *Format : Heatmap* transforme les buckets `le` en tranches ; c'est la seule visualisation qui montre une distribution qui bouge dans le temps, et c'est le moment de lancer `./lab.sh chaos latency on` pour voir la tache orange monter.

Étape 3 — Row « Détail » : table avec transformations, bar chart, info 10 min

Énoncé.

- Top routes** — Table avec deux requêtes Instant / Table : débit par route et p95 par route. Transformations *Merge → Organize fields* (renommer `Value #A → req/s`, `Value #B → p95`, masquer Time) → *Sort by req/s décroissant*. Overrides : unité reqps sur req/s ; unité s, 3 décimales et *Cell options → Cell type : Colored background* avec seuils 0.5 / 1 sur p95.
- Requêtes par code HTTP** — Bar chart : `sum by (status) (increase(http_requests_total{...}[$__range]))` en Instant, transformation *Reduce (mode Series to rows, calcul Last)*.

12. Version déployée — Stat en *Text mode* : *Name*, requête `shop_app_info{...}` avec légende

```
{{instance_name}} v{{version}} .
```

Corrigé. Le *Colored background* sur une colonne de table est le moyen le plus efficace de faire ressortir un problème dans une liste. Le *Reduce* → *Series to rows* transforme N séries en N lignes pour un bar chart par catégorie ; c'est la transformation à connaître pour les histogrammes de catégories.

Étape 4 — Annotations et liens 5 min

Énoncé.

13. Barre latérale *Add* → *Annotation query* : nom `Chaos`, source Prometheus, requête

```
changes(shop_chaos_mode{instance=~"$instance"}[1m]) > 0
```

, titre `Chaos {{mode}}`, couleur rouge.

14. *Add* → *Link* : un lien vers le dashboard *TP 4 - Serveur Linux* (type Dashboard, *Keep time range*), et un lien externe vers Prometheus.

15. Lancez `./lab.sh chaos errors on` puis `off` deux minutes plus tard : les traits rouges apparaissent sur tous les graphiques.

Corrigé. Une annotation à partir d'une requête Prometheus est une requête qui renvoie des séries aux instants où l'événement a lieu ; `changes(...) > 0` isole les bascules. En production, on annote les déploiements (via l'API `/api/annotations` depuis la CI) : c'est ce qui permet de répondre tout de suite à « ça a commencé après le déploiement de 14h12 ? ».

Étape 5 — Dashboards as code 10 min

Énoncé.

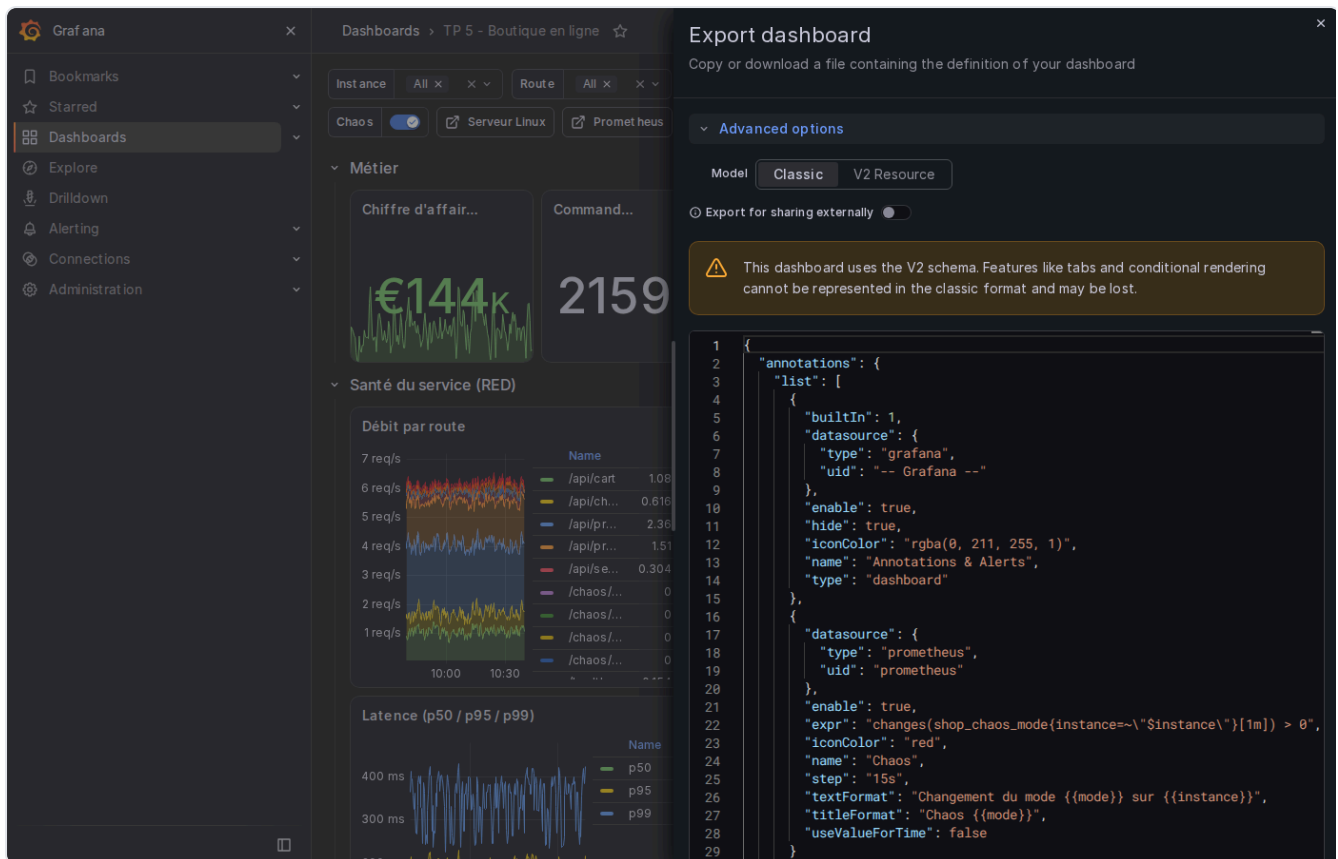
16. Bouton *Export* (barre du haut) → *Export as code*. Dépliez *Advanced options* : *Model* : *Classic* (le choix JSON/YAML n'existe que pour V2 Resource). Laissez *Share dashboard with another instance* désactivé. *Copy to clipboard* (ou *Download file*).

17. Enregistrez le JSON dans `grafana/dashboards/tp5-boutique.json`. Attendez 10 secondes (le provider relit le dossier) et rechargez la liste des dashboards : un second « TP 5 » avec un cadenas est apparu (provisionné). Supprimez la version manuelle ou renommez-la.

18. Modifiez un titre de panel dans le JSON. Que se passe-t-il ? Et si vous modifiez le dashboard provisionné dans l'interface et sauvegardez ?

Corrigé. Le provider `grafana/provisioning/dashboards/dashboards.yml` a `allowUiUpdates: true`, donc on peut sauvegarder depuis l'interface, mais la prochaine modification du fichier écrasera. En production, on met `allowUiUpdates: false` et `disableDeletion: true` : la seule source de vérité est Git. Avec un `uid` fixe dans le JSON, l'URL du dashboard ne change jamais, même après suppression/recréation : indispensable pour les liens et les runbooks.

Sur les deux modèles d'export : *Classic* est le JSON historique (celui des dashboards communautaires et de nos fichiers), *V2 Resource* est le nouveau format « à la Kubernetes » (`apiVersion: dashboard.grafana.app/v2`) qu'utilisent Git Sync et la nouvelle API `/apis`. Le provisioning par fichier accepte les deux, mais je fais travailler en *Classic* : plus lisible, compatible avec tout l'outillage existant (Grafonnet, grafanalib, Terraform). L'option *Share dashboard with another instance* remplace la source de données par une variable `${DS_PROMETHEUS}` et ajoute un bloc `__inputs` : c'est le format pour publier sur grafana.com, pas pour provisionner.



La boîte d'export de Grafana 13 : Model Classic ou V2 Resource

Git Sync (Grafana 13, *Administration* → *Provisioning*) va plus loin : Grafana lit et écrit les dashboards directement dans un dépôt GitHub/GitLab, avec pull request. Je le montre en capture d'écran seulement (il faut un dépôt et un token).

Module 10 — Provisioning, utilisateurs, droits, éditions 30 min

Objectif. Savoir tout provisionner par fichier, organiser les droits proprement, et répondre à « faut-il acheter Enterprise ? ».

Ce que je dis

Tout se provisionne. Sources de données, dashboards, alerting (contact points, politiques, règles), plugins, et même les dossiers, par fichiers YAML/JSON dans `/etc/grafana/provisioning/`. On l'a fait pour la source Prometheus et le dashboard d'accueil. Alternative : Terraform (provider Grafana), ou l'API HTTP. Règle : en production, personne n'a le droit d'éditer à la main ce qui est provisionné.

Le modèle de droits.

- **Organisation** : cloisonnement total (sources de données, dashboards, utilisateurs). Une organisation par client chez un hébergeur, sinon une seule. Ne pas utiliser les orgs pour séparer des équipes : les dossiers suffisent.
- **Utilisateurs** : locaux, ou via OAuth (GitHub, Google, Azure AD/Entra, Okta), LDAP, SAML (Enterprise). Rôle par organisation : *Admin*, *Editor*, *Viewer* (et *None* depuis la 10). Le *Server Admin* est à part : il gère les orgs et les utilisateurs.

- **Teams** : des groupes. On donne les droits aux teams, pas aux personnes.
- **Permissions sur dossiers et dashboards** : View / Edit / Admin, par utilisateur, team ou rôle. Un Viewer peut être Editor sur un dossier précis.
- **Service accounts** : des comptes techniques avec un token, pour les scripts et la CI. Ils remplacent les API keys.
- **Permissions sur les sources de données** (Enterprise/Cloud) : limiter qui peut interroger quoi.
RBAC fin (rôles personnalisés) : Enterprise aussi.

Bonne pratique : dossier par équipe, team par équipe, team = Editor sur son dossier, Viewer ailleurs.
Dashboards provisionnés en lecture seule dans un dossier « Officiel ».

OSS contre Enterprise (et Cloud).

	OSS (AGPL v3)	Enterprise	Cloud
Dashboards, alerting, Explore, provisioning, Git Sync	Oui	Oui	Oui
Sources de données	Toutes les open source (Prometheus, Loki, SQL, Elastic, ...)	+ connecteurs commerciaux (Splunk, Datadog, ServiceNow, Oracle, SAP HANA, ...)	idem
Authentification	OAuth, LDAP, proxy	+ SAML, SCIM, sync de teams	idem
Droits	Rôles fixes, permissions dossiers	+ RBAC fin, permissions par source de données	idem
Rapports PDF planifiés, white-labelling, audit logs, query caching, usage insights	Non	Oui	Oui
Support	Communauté	Contrat	Inclus
Grafana Assistant (IA)	Preview, via connexion Cloud	Preview	Oui
Prix	Gratuit	Licence par utilisateur actif	Free tier puis à l'usage

Ma réponse : la plupart des entreprises n'ont pas besoin d'Enterprise. On y va pour SAML/SCIM imposés par la sécurité, pour un connecteur commercial, ou pour les rapports. Grafana Cloud est intéressant pour ne pas opérer Prometheus/Loki/Tempo soi-même (c'est Mimir derrière).

Ce que je montre

Administration → Users and access → Users / Teams / Service accounts, puis sur un dossier :
Dashboards → dossier Formation → Folder actions → Manage permissions.

Exercice 2.30 — Un lecteur 5 min

Énoncé. Créez l'utilisatrice `lecture` (Administration → Users and access → Users → New user), rôle *Viewer*. Ouvrez une fenêtre de navigation privée, connectez-vous avec ce compte. Que peut-elle faire ? Essayez de modifier le TP 5.

Corrigé. Elle voit tout, modifie rien, n'a pas Explore (par défaut les Viewers n'ont pas Explore ; `viewers_can_edit` a été retiré en Grafana 12, il faut des permissions par dossier). Le bouton *Edit* est absent.

Exercice 2.31 — Une team éditrice sur son dossier 10 min

Énoncé.

1. Créez un dossier *Boutique* et déplacez-y le dashboard TP 5 (*Move*).
2. Créez la team *equipe-boutique* (Administration → Users and access → Teams → New team), ajoutez `lecture`.
3. Sur le dossier Boutique : *Folder actions* → *Manage permissions* → *Add a permission* → *Team equipe-boutique* → *Edit*.
4. En navigation privée, avec `lecture` : elle peut maintenant éditer le TP 5 mais pas le TP 4.

Corrigé. C'est le modèle à répliquer : rôle global minimal, droits élevés par dossier via team. Question : « et si on veut qu'elle voie le dossier Boutique mais pas le dossier Formation ? » → retirer le rôle *Viewer* (rôle *None*) et donner *View* sur Boutique uniquement.

Exercice 2.32 — Une organisation 5 min

Énoncé. Administration → General → Organizations → New org `Client-B`. Basculez dedans (menu profil en bas / haut). Que voyez-vous ?

Corrigé. Rien : pas de source de données, pas de dashboard. Le provisioning ne s'applique qu'à `orgId: 1` sauf mention contraire. Retour à *Main Org.*. Message : les orgs, c'est pour le multi-tenant strict, et c'est lourd.

Exercice 2.33 — Service account et API (bonus, 5 min)

Énoncé. Créez un service account `ci` (rôle *Editor*), générez un token, puis depuis un terminal listez les dashboards :

```
curl -s -H "Authorization: Bearer <token>" http://localhost:3000/api/search?type=dash-db
```

Bonus : posez une annotation « Déploiement v1.4.3 » via `POST /api/annotations`.

Corrigé.

```
curl -s -X POST http://localhost:3000/api/annotations \
-H "Authorization: Bearer <token>" -H "Content-Type: application/json" \
-d '{"tags":["deploy"],"text":"Déploiement v1.4.3"}'
```

L'annotation apparaît sur tous les dashboards dont la source d'annotations intégrée (*Annotations & Alerts*) est active, sur le trait de temps courant. En CI, c'est une étape de quelques lignes dans le pipeline. Remarque : l'API historique `/api/...` reste fonctionnelle en 13 mais Grafana la fait

progressivement migrer vers `/apis/...` (API à la Kubernetes) ; pour les scripts, les deux marchent aujourd'hui.

17h20 — Récap et quiz du jour 2 10 min

1. Que renvoie `rate()` et sur quel type de métrique ? *(un débit par seconde ; counter uniquement)*
2. Pourquoi `sum by (le)` dans `histogram_quantile` ? *(la fonction a besoin des buckets)*
3. `$_rate_interval`, ça sert à quoi ? *(la fenêtre de rate adaptée au zoom)*
4. Multi-valeur + All : `=` ou `=~` ? *(=~)*
5. Comment afficher « UP » au lieu de 1 ? *(value mapping)*
6. Comment fusionner deux requêtes dans une table ? *(transformation Merge)*
7. Où met-on un dashboard pour qu'il survive à une réinstallation ? *(dans Git, provisionné)*
8. Un Viewer peut-il éditer un dashboard ? *(oui, avec une permission Edit sur son dossier)*
9. Trois choses qu'apporte Enterprise. *(SAML/SCIM, RBAC fin, connecteurs commerciaux, rapports...)*

État attendu ce soir : `recording.yml` rempli, les dashboards TP 4 et TP 5 sauvegardés dans le dossier Formation, `tp5-boutique.json` dans `grafana/dashboards/`. Demain, on casse tout et on se fait prévenir.

Jour 3 — Réagir : alertes, exploitation, passage à l'échelle

Objectif de la journée : le matin, les stagiaires écrivent des alertes qui ne réveillent que pour de bonnes raisons et les routent vers Teams, Slack et une boîte de réception. L'après-midi, alerting Grafana, exploitation de Prometheus (performance, sauvegarde, sécurité), passage à l'échelle, et un war game pour finir.

Heure	Séquence	Durée
9h00	Rappel du jour 2, quiz flash	10 min
9h10	Module 11 — Philosophie de l'alerting et règles Prometheus	30 min
9h40	Module 12 — Alertmanager	25 min
10h05	TP 6 — Alertes Prometheus et routage Alertmanager	55 min
11h00	Pause	15 min
11h15	Module 13 — Notifications tierces : Slack, PagerDuty, Teams (Workflows), GitHub, templates	30 min
11h45	TP 7 — Cas pratique : alerte CPU → Teams	45 min
12h30	Déjeuner	
14h00	Module 14 — Alerting Grafana	25 min
14h25	TP 8 — Alerting Grafana, de l'interface au code	45 min
15h10	Module 15 — Performances, limites, bonnes pratiques + exercices 3.1 à 3.5	35 min
15h45	Pause	15 min
16h00	TP 9 — Sauvegarde, restauration, remote write, sécurité	45 min
16h45	Module 16 — Mise à l'échelle et écosystème	20 min
17h05	War game + évaluation finale	25 min

9h00 — Rappel 10 min

Quiz flash. Vérification que `recording.yml` est en place chez tout le monde (les alertes du matin s'appuient dessus pour certaines) et que le dashboard TP 5 est là (on va le regarder pendant les pannes).

Module 11 — Philosophie de l'alerting et règles Prometheus 30 min

Objectif. Savoir ce qui mérite une alerte, et écrire une règle propre.

Ce que je dis

La règle de Google SRE. On alerte sur les **symptômes**, pas sur les causes. « CPU à 90 % » est une cause possible d'un problème... ou le signe que le serveur fait son travail. « Les clients attendent plus de 3 secondes » ou « 5 % des paiements échouent » sont des symptômes : quelqu'un souffre, il faut agir. Corollaire : une alerte doit être **actionnable**. Si la réponse à « qu'est-ce que je fais quand ça sonne ? » est « rien, je regarde », ce n'est pas une alerte, c'est un panel de dashboard.

Anecdote — les 400 alertes. Une équipe d'astreinte recevait 400 notifications par jour. Disques à 80 %, CPU à 85 %, un ping raté. Résultat : plus personne ne les lisait, et la nuit où la base de données est tombée, l'alerte est passée avec les autres. Après nettoyage : 12 alertes, toutes sur des symptômes clients, chacune avec un runbook. La fatigue d'alerte tue plus de systèmes que les pannes.

Les niveaux. Pas besoin de plus de trois : `critical` (on réveille quelqu'un), `warning` (on regarde demain matin), `info` (ticket, tableau de bord). La sévérité est un label, on routera dessus.

Anatomie d'une règle.

```
groups:
- name: shop-api
  rules:
  - alert: ShopHighErrorRate           # nom en CamelCase, unique
    expr: |                           # alerte si l'expression renvoie des séries
      sum by (instance) (rate(http_requests_total{job="shop-api", status=~"5.."}[5m]))
      / sum by (instance) (rate(http_requests_total{job="shop-api"}[5m])) > 0.05
    for: 2m                           # vrai pendant 2 min avant FIRING
    keep_firing_for: 3m               # reste FIRING 3 min après le retour à la normale
    labels:
      severity: critical
      team: boutique
    annotations:
      summary: "Taux d'erreur élevé sur {{ $labels.instance }}"
      description: "{{ $value | humanizePercentage }}" des requêtes échouent (seuil 5 %).
      runbook_url: "https://.../shop-errors.md"
```

Points clés :

- Une alerte se déclenche **par série renvoyée** : avec `by (instance)`, deux instances en erreur = deux alertes. Sans `by`, une seule alerte globale. C'est un choix.
- Cycle de vie : *inactive* → *pending* (condition vraie, `for` pas écoulé) → *firing*. `for` évite les alertes sur un pic d'une seconde. `keep_firing_for` (depuis 2.42) évite le clignotement quand la valeur oscille autour du seuil.
- *Labels* : servent à router (Alertmanager) et à grouper. *Annotations* : servent aux humains, avec des templates Go : `{{ $labels.x }}`, `{{ $value }}`, filtres `humanize`, `humanizePercentage`, `humanizeDuration`, `printf "%.2f"`.
- `runbook_url` est une convention (pas un mot-clé), reprise par Alertmanager et Grafana dans leurs templates.

Les classiques à avoir. `up == 0` (mais attention aux blackbox), `probe_success == 0`, `prometheus_config_last_reload_successful == 0`, `absent(up{job="critique"})`, le taux d'erreur, le p95, `predict_linear` sur le disque, le batch trop vieux, et `prometheus_notifications_errors_total`

(Prometheus n'arrive plus à parler à Alertmanager : la seule alerte qui ne peut pas être envoyée, à surveiller autrement).

Tester. `promtool check rules`, puis `promtool test rules` avec des séries simulées, comme au TP 3. Et en conditions réelles avec le chaos.

Ce que je montre

Dans Prometheus, **Alerts** : la règle `TargetDown` déjà présente. J'arrête `shop-api-2`, on regarde passer *pending* puis *firing* au bout d'une minute, la notification arrive dans l'Inbox. Je redémarre, *resolved* arrive.

Module 12 — Alertmanager 25 min

Objectif. Comprendre l'arbre de routage, le regroupement, l'inhibition et les silences.

Ce que je dis

Son rôle. Prometheus évalue et envoie ; Alertmanager décide *qui prévenir, quand, et combien de fois*. Il déduplique (deux Prometheus en HA envoient la même alerte), il regroupe (50 instances down = 1 notification), il route (critical → astreinte, boutique → canal boutique), il inhibe (si la base est down, taire les alertes des services qui en dépendent), il respecte les silences et les plages horaires.

L'arbre de routage.

```
route:                                # la racine reçoit tout
  receiver: inbox-default
  group_by: ["alertname", "job"]
  group_wait: 30s                     # regrouper avant la 1re notification
  group_interval: 5m                  # avant d'envoyer les nouveautés d'un groupe
  repeat_interval: 4h                 # re-notifier une alerte toujours active
  routes:
    - matchers: [severity = critical]
      receiver: astreinte-teams
      continue: true                  # évaluer aussi les routes suivantes
    - matchers: [team = boutique]
      receiver: boutique-slack
```

Une alerte descend l'arbre : première route qui matche, elle s'y arrête, sauf `continue: true`. Les routes enfants héritent des paramètres du parent. Les `matchers` acceptent `=`, `!=`, `=~`, `!~`.

Le regroupement est le concept qui surprend. `group_by: [alertname, job]` : toutes les alertes `TargetDown` du job `shop-api` partent dans **une** notification. `group_wait` retarde la première pour laisser arriver les autres. C'est ce qui évite les 50 mails à 3h du matin. Sans `group_by` (ou `group_by: ['...']`), une notification par alerte.

L'inhibition. `inhibit_rules` : si une alerte *source* est active (`TargetDown` sur l'instance X), on tait les alertes *cibles* (`ShopHighErrorRate` sur la même instance X grâce à `equal: [instance]`). Quand le serveur est éteint, inutile de dire qu'il répond lentement.

Les silences. Créés dans l'interface (ou `amtool silence add`), avec des matchers et une durée. Pour une maintenance planifiée. Ils s'appliquent après le routage. Les **time intervals** (`mute_time_intervals`, `active_time_intervals`) sont des silences récurrents : week-end, nuit, heures ouvrées.

Les receivers. Un nom, et une ou plusieurs intégrations : `webhook_configs`, `slack_configs`, `msteamsv2_configs`, `pagerduty_configs`, `opsgenie_configs`, `email_configs`, `telegram_configs`, `discord_configs`, `webex_configs`, `pushover_configs`, `sns_configs`, `victorops_configs`, `mattermost_configs` ... Le webhook est le joker : n'importe quel service HTTP peut recevoir le JSON.

Le format webhook. C'est celui que reçoit notre Inbox :

```
{ "receiver": "inbox-default", "status": "firing", "groupLabels": {"alertname": "TargetDown"},
  "commonLabels": {...}, "commonAnnotations": {...}, "externalURL": "http://localhost:9093",
  "alerts": [ { "status": "firing", "labels": {...}, "annotations": {...},
    "startsAt": "...", "endsAt": "...", "generatorURL": "...", "fingerprint": "..." }
] }
```

Haute disponibilité. Plusieurs Alertmanager en cluster (gossip) ; chaque Prometheus envoie à tous ; ils dédupliquent entre eux. Quelques lignes de configuration, on en parle au module 16.

Ce que je montre

- `http://localhost:9093` : les alertes actives, les groupes, le bouton *Silence*, **Status** avec la configuration chargée.

Alertmanager Alerts Silences Status Settings Documentation [New Silence](#)

Filter Group Receiver: All ☐ Silenced ☐ Inhibited ☐ Muted

Custom matcher, e.g. `env="production"`

+ Expand all groups

- + boutique-slack alertname="ShopCheckoutSlow" + 1 alert
- + astreinte-teams alertname="ShopHighErrorRate" + 2 alerts
- + boutique-slack alertname="ShopHighErrorRate" + 2 alerts
- + astreinte-teams alertname="ShopNoOrders" + 1 alert
- + boutique-slack alertname="ShopNoOrders" + 1 alert
- + astreinte-teams alertname="BlackboxProbeFailed" + job="blackbox-http" + 1 alert

Alertmanager : alertes groupées par receiver

Boîte de réception des notifications

Rafraîchissement automatique toutes les 5 s. 5 message(s). [Vider](#) - [JSON](#)

slack 2026-09-26 08:51:00 · /slack/boutique
[FIRING] ShopNoOrders • Plus aucune commande depuis 10 minutes malgré du trafic — Le tunnel de commande est probablement cassé.
 ▶ [JSON brut](#)

teams 2026-09-26 08:50:40 · /teams/astreinte
[FIRING:1] ShopNoOrders (shop-api-1:5000 critical boutique)
 # Alerts Firing: Labels: - alertname = ShopNoOrders - instance = shop-api-1:5000 - severity = critical - team = boutique Annotations: - description = Le tunnel de commande est probablement cassé. - summary = Plus aucune commande depuis 10 m...
 ▶ [JSON brut](#)

webhook/grafana 2026-09-26 08:48:20 · /webhook/grafana
[RESOLVED] DatasourceNoData severity=warning instance=- | CPU à 0 % sur [no value]
 ▶ [JSON brut](#)

teams 2026-09-26 08:48:05 · /teams/grafana
[FIRING] Boutique - taux d'erreur élevé (Grafana)
 Firing Value: A=29.245283018867926, B=29.245283018867926, C=1 Labels: - alertname = Boutique - taux d'erreur élevé (Grafana) - grafana_folder = Formation - instance = shop-api-1:5000 - severity = critical - source = grafana - team = bou...
 ▶ [JSON brut](#)

teams 2026-09-26 08:47:17 · /teams/astreinte
[FIRING:2] ShopHighErrorRate (formation prom-1 critical boutique)
 # Alerts Firing: Labels: - alertname = ShopHighErrorRate - cluster = formation - instance = shop-api-1:5000 - replica = prom-1 - severity = critical - team = boutique Annotations: - description = 23.3% des requêtes échouent (seuil 5 %). - r...
 ▶ [JSON brut](#)

L'Inbox du lab : une carte Teams, deux messages Slack

- `docker compose exec alertmanager amtool alert` (les alertes actives ; `amtool` trouve l'URL dans `/etc/amtool/config.yml`, monté par le compose), puis `docker compose exec alertmanager amtool config routes show --config.file=/etc/alertmanager/alertmanager.yml` (l'arbre en ASCII) et ...
`amtool config routes test --config.file=/etc/alertmanager/alertmanager.yml severity=critical team=boutique` (« vers quel receiver ? »). Le `--config.file` est obligatoire pour ces deux commandes.

TP 6 — Alertes Prometheus et routage Alertmanager 55 min

Objectif. Écrire quatre alertes métier/service, les tester en cassant la boutique, puis construire l'arbre de routage avec regroupement, inhibition, silence et plage horaire.

Mise en situation. L'équipe boutique veut être prévenue dans son canal quand le site souffre.

L'astreinte veut recevoir uniquement le critique, tout de suite. L'infra ne veut rien recevoir le week-end sauf le critique. Et personne ne veut recevoir « erreurs sur shop-api-2 » quand shop-api-2 est éteint.

Partie 1 — Les alertes de la boutique 15 min

Énoncé. Dans `prometheus/rules/alerts.yml`, groupe `shop-api`, écrivez :

1. `ShopHighErrorRate` (critical, team boutique) : taux d'erreur `5xx > 5 %` par instance pendant 2 min, `keep_firing_for: 3m`, `summary` avec l'instance, `description` avec la valeur en pourcentage (`humanizePercentage`), `runbook_url` vers `docs/runbooks/shop-errors.md`.
2. `ShopCheckoutSlow` (warning, boutique) : `p95 de /api/checkout > 1 s` pendant 3 min, `summary` avec la valeur (`humanizeDuration`).

3. `ShopNoOrders` (critical, boutique) : aucune commande depuis 10 min **alors qu'il y a du trafic** (deux conditions avec `and`), pendant 5 min.
4. `ShopStockLow` (info, team logistique) : stock < 15 unités, avec le produit dans le summary. Puis dans un groupe `disponibilite`, ajoutez `BlackboxProbeFailed` (critical) sur `probe_success == 0`. Validez, testez (`./lab.sh test` doit toujours passer), rechargez. Vérifiez dans **Alerts**.

Corrigé. Fichier complet : `solutions/jour-3/alerts.yml`. Les points à discuter :

- `ShopNoOrders` : `sum(rate(shop_orders_total[10m])) == 0 and sum(rate(http_requests_total{job="shop-api"}[10m])) > 0`. Le `and` apparie des vecteurs sans labels (les deux `sum` n'en ont aucun) : ça marche. Sans le `and`, l'alerte sonnerait aussi la nuit quand il n'y a simplement personne sur le site.
- `ShopStockLow` sonnera peut-être pendant le TP (le stock descend puis se réassortit) : c'est volontaire, ça fait vivre l'Inbox.
- `BlackboxProbeFailed` : ceux qui n'ont pas Internet ont déjà `https://prometheus.io` en échec ; ça leur fait une alerte critique de test gratuite.

Partie 2 — Casser et vérifier 10 min

Énoncé.

1. `./lab.sh chaos errors on`. Chronométrez : au bout de combien de temps `ShopHighErrorRate` passe *pending* ? *firing* ? À quel moment la notification arrive dans l'Inbox ? Expliquez chaque délai.
2. `./lab.sh chaos errors off`. Combien de temps avant *resolved* ? Pourquoi ?
3. Même exercice avec `./lab.sh chaos latency on` et `ShopCheckoutSlow`.
4. Regardez le dashboard TP 5 pendant ce temps : les annotations Chaos et le panneau d'erreurs.

Corrigé. Pending dès que `rate[5m]` dépasse 5 %, soit environ une minute après le chaos (la fenêtre de 5 min se remplit d'erreurs progressivement ; à 40 % d'erreurs, la barre des 5 % est franchie au bout de 40 s environ, plus un scrape et une évaluation). Firing 2 min plus tard (`for`). Notification 30 s plus tard (`group_wait`). Au retour : la fenêtre de 5 min met plusieurs minutes à redescendre sous 5 %, puis `keep_firing_for: 3m`, puis la notification *resolved* au prochain `group_interval`. Total : 8 à 10 minutes. Leçon : chaque paramètre a un coût en réactivité, et l'empilement `[5m] + for + keep_firing_for + group_wait` s'additionne.

Partie 3 — L'arbre de routage 15 min

Énoncé. Dans `alertmanager/alertmanager.yml` :

1. Les receivers vers l'Inbox : `inbox-default` (`/webhook/default`), `infra-inbox` (`/webhook/infra`), `astreinte-teams` en `msteamsv2_configs` vers `http://inbox:8080/teams/astreinte` (on mettra une vraie URL Teams au TP 7), `boutique-slack` en `slack_configs` vers `http://inbox:8080/slack/boutique`, `canal` `#boutique-alertes`.
2. Routes : `severity = critical` → `astreinte-teams`, `group_wait 10 s`, `repeat_interval 1 h`, `continue: true` ; `team = boutique` → `boutique-slack` ; `severity = info` → `inbox-default` avec `group_interval 30 m` et `repeat_interval 24 h` ; `team = infra` → `infra-inbox`.
3. Validez (`./lab.sh check`), rechargez, puis `docker compose exec alertmanager amtool config routes test --config.file=/etc/alertmanager/alertmanager.yml severity=critical team=boutique` : vers quels receivers ?

4. Relancez `./lab.sh chaos errors on` et observez l'Inbox : combien de messages, sur quels canaux, avec quel format ?

Corrigé. Fichier : `solutions/jour-3/alertmanager.yml` (routes 1 à 3 et 5). Une alerte critical + boutique part vers `astreinte-teams` **et** `boutique-slack` grâce à `continue: true`. Dans l'Inbox, on compare le JSON Slack (`attachments` avec `color: danger`) et la carte adaptative Teams (`attachments[].content.body[]` de type `TextBlock`) : deux formats générés par Alertmanager à partir de la même alerte.

`amtool config routes test` est peu connu et évite des heures de « pourquoi ça n'arrive pas dans le bon canal ».

Partie 4 — Inhibition 5 min

Énoncé. Ajoutez deux `inhibit_rules` : (a) `TargetDown` inhibe `ShopHighErrorRate` et `ShopCheckoutSlow` sur la même `instance` ; (b) une alerte `critical` inhibe le `warning` de même `alertname` sur la même `instance` (règle générique : aucune alerte du lab n'existe en deux sévérités, elle ne se déclenchera pas ici). Testez : `chaos errors on`, puis `docker compose stop shop-api-2`. Que voit-on dans Alertmanager ?

Corrigé. L'alerte `ShopHighErrorRate{instance="shop-api-2:5000"}` passe *inhibited* dans l'interface (filtre *Inhibited*). Elle reste visible mais n'est plus notifiée. Redémarrer `shop-api-2` et remettre le chaos à off.

Partie 5 — Silence et plage horaire 10 min

Énoncé.

1. Dans l'interface Alertmanager, créez un silence de 30 min sur `alertname="ShopStockLow"` avec un commentaire. Vérifiez que l'alerte n'est plus notifiée. Même chose en ligne de commande : `docker compose exec alertmanager amtool silence add alertname=ShopStockLow -d 30m -c "réassort en cours"` puis `amtool silence query` et `amtool silence expire <id>`.
2. Ajoutez un `time_intervals` nommé `nuit-et-weekend` (samedi, dimanche, et 20h-8h en Europe/Paris) et appliquez-le en `mute_time_intervals` sur la route `team = infra`.

Corrigé. Le piège habituel : un intervalle `20:00 → 08:00` est refusé (« start time cannot be equal or greater than end time »). Il faut deux intervalles : `20:00-24:00` et `00:00-08:00`. Grafana a exactement la même contrainte. Deuxième piège : `location` doit être un nom de fuseau IANA, et le conteneur doit avoir la base `tzdata` (l'image officielle l'a).

Ce que je vérifie. `./lab.sh check` passe chez tout le monde, l'Inbox a reçu au moins un message Teams et un message Slack, et le chaos est off.

Module 13 — Notifications tierces et ChatOps 30 min

Objectif. Configurer Slack, PagerDuty, Teams via Workflows, un webhook vers GitHub, et personnaliser les messages.

Ce que je dis

Slack. Deux méthodes : le *incoming webhook* historique (`api_url` , une URL secrète par canal) ou une application Slack avec token (`slack_configs` supporte `api_url` et, depuis Alertmanager 0.30, une configuration « app » avec token de bot). Dans les deux cas, on met le secret dans un fichier (`api_url_file`) et jamais dans le YAML commité.

```
slack_configs:
- api_url_file: /etc/alertmanager/secrets/slack.url
  channel: "#boutique-alertes"
  send_resolved: true
  title: '{{ .Status | toUpper }} [{{ .CommonLabels.alertname }}]'
  text: '{{ range .Alerts }}• {{ .Annotations.summary }}\n' {{ end }}
```

PagerDuty. L'outil d'astreinte de référence (avec Opsgenie, en fin de vie chez Atlassian, et Grafana OnCall/IRM). Events API v2 : une *integration key* par service.

```
pagerduty_configs:
- routing_key_file: /etc/alertmanager/secrets/pagerduty.key
  severity: '{{ .CommonLabels.severity }}'
  description: '{{ .CommonLabels.alertname }}: {{ .CommonAnnotations.summary }}
```

PagerDuty gère l'escalade, les plannings, l'acquittement ; Alertmanager lui envoie *trigger* et *resolve* avec la même clé de déduplication.

Microsoft Teams, la méthode 2026. Les connecteurs Office 365 (« Incoming Webhook » dans les paramètres d'un canal) ont été retirés par Microsoft : création bloquée depuis août 2024, coupure définitive des connecteurs existants en mai 2026, après plusieurs reports. L'ancien `msteams_configs` d'Alertmanager est donc déprécié. La méthode actuelle passe par **Workflows** (Power Automate) :

1. Dans Teams, dans le canal cible : ... → *Workflows* → modèle « **Post to a channel when a webhook request is received** » (« Publier dans un canal lorsqu'une demande de webhook est reçue »).
2. Donner un nom, choisir l'équipe et le canal, créer. Teams affiche une URL `https://prod-xx.westeurope.logic.azure.com:443/workflows/.../triggers/manual/paths/invoke?...`. La copier, c'est le secret.
3. Le flux attend une *Adaptive Card* dans le corps du POST. Alertmanager (`msteamsv2_configs`) et Grafana (contact point *Microsoft Teams*) la produisent tout seuls.

```
msteamsv2_configs:
- webhook_url_file: /etc/alertmanager/secrets/teams.url
  send_resolved: true
```

Pour un canal privé, il faut éditer le flux et passer l'action « Post your own adaptive card as the Flow bot » de *Flow bot* à *User*. Et si Grafana dit « envoyé » mais que rien n'arrive, on regarde l'historique d'exécution du flux dans Power Automate.

Notre Inbox expose `/teams/...` et accepte la carte : on peut tout tester sans compte Microsoft. Le jour où on a la vraie URL, on la remplace, rien d'autre ne change.

GitHub. Le pattern ChatOps : une alerte ouvre une issue (ou commente celle qui existe) et la ferme à la résolution. On ne branche pas Alertmanager directement sur l'API GitHub : on passe par un *repository_dispatch* qui déclenche un workflow Actions. Le dépôt contient `.github/workflows/alert-`

`to-issue.yml` qui fait exactement ça. Côté émetteur, Grafana (contact point *Webhook*, payload personnalisé et en-tête `Authorization: Bearer <token>`) est le plus simple ; Alertmanager sait aussi le faire avec `webhook_configs` + `http_config.authorization` + le bloc `payload` (Alertmanager ≥ 0.32). Alternative très répandue : un petit relais (une fonction serverless, un `n8n`) entre les deux.

Les templates. Alertmanager utilise les templates Go. On définit ses blocs dans un fichier `.tmpl` chargé par `templates:` et on les appelle avec `{{ template "nom" . }}`. Les variables utiles : `.Status`, `.Alerts` (et `.Alerts.Firing`, `.Alerts.Resolved`), `.CommonLabels`, `.CommonAnnotations`, `.GroupLabels`, `.ExternalURL`. Notre `alertmanager/templates/formation.tmpl` en contient deux. Ce qu'un bon message contient : le statut, le nombre d'alertes, le résumé, la sévérité, l'instance, le lien runbook, le lien vers le dashboard. Ce qu'il ne contient pas : un dump de tous les labels.

Grafana IRM. Pour mémoire : Grafana propose sa propre astreinte (plannings, escalades, appels) dans Grafana Cloud ; la version OSS d'OnCall a été archivée en mars 2026.

Ce que je montre

- La création du flux Workflows, pas à pas (annexe E) ; en direct si j'ai un tenant de démo.
- Le fichier `.github/workflows/alert-to-issue.yml` et un `curl` de test si le token est disponible.

TP 7 — Cas pratique : alerte CPU élevé, notification dans Teams

45 min

Objectif. Le cas pratique du programme, de bout en bout : la règle, le routage, la notification Teams via Workflows, un message personnalisé.

Mise en situation. L'équipe infra veut être prévenue dans son canal Teams quand un serveur dépasse 80 % de CPU pendant plus de deux minutes, avec un message lisible : le serveur, la valeur, un lien vers le dashboard.

Partie 1 — La règle 10 min

Énoncé. Groupe `infrastructure` dans `alerts.yml` :

1. `HostHighCpuLoad` (warning, team infra) : CPU > 80 % pendant 2 min, fenêtre de `rate` 2 min, `summary` : « CPU à NN % sur » (`printf "%.0f"`).
2. Bonus : `HostOutOfMemory` (mémoire > 90 % pendant 5 min), `HostDiskWillFillIn24h` (`predict_linear` sur 6 h, et espace < 20 %), `BackupTooOld` (dernière sauvegarde Pushgateway > 24 h), `PrometheusConfigReloadFailed`.
3. Ajoutez un test unitaire pour `HostHighCpuLoad` dans `prometheus/tests/` : une série `node_cpu_seconds_total{mode="idle", cpu="0", instance="srv"}` qui n'augmente plus (CPU saturé), et vérifiez l'alerte à 4 min. Validez, testez, rechargez.

Corrigé. Règle dans `solutions/jour-3/alerts.yml`. Le test :

```
rule_files:
  - ../rules/alerts.yml
evaluation_interval: 15s
tests:
  - interval: 15s
```

```

input_series:
  # idle n'augmente plus : le CPU est occupé à 100 %
  - series: 'node_cpu_seconds_total{mode="idle", cpu="0", instance="srv", job="node"}'
    values: "100 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100 100"
alert_rule_test:
  - eval_time: 4m
    alertname: HostHighCpuLoad
    exp_alerts:
      - exp_labels:
          severity: warning
          team: infra
          instance: srv
        exp_annotations:
          summary: "CPU à 100 % sur srv"
          description: "L'utilisation CPU dépasse 80 % depuis 2 minutes."

```

Attention à la subtilité : `rate` d'une série constante vaut 0, donc $1 - 0 = 100\%$. Les valeurs doivent être présentes (pas de trous) sur toute la fenêtre.

Partie 2 — Le routage vers Teams 10 min

Énoncé.

1. Ajoutez une route `alertname = HostHighCpuLoad` → receiver `astreinte-teams`, `group_wait: 10s`, **avant** la route `team = infra` (pourquoi ?).
2. Si vous avez une URL Workflows : remplacez `webhook_url` par la vraie URL (ou mieux : `webhook_url_file` pointant sur un fichier monté). Sinon, gardez `http://inbox:8080/teams/astreinte`.
3. Validez, rechargez, `docker compose exec alertmanager amtool config routes test -- config.file=/etc/alertmanager/alertmanager.yml alertname=HostHighCpuLoad team=infra severity=warning`.

Corrigé. L'ordre compte : l'arbre s'arrête à la première route qui matche (sans `continue`). Si `team = infra` est avant, l'alerte part vers `infra-inbox` (et se fait muter le week-end). `amtool config routes test` doit répondre `astreinte-teams`.

Partie 3 — Déclencher 10 min

Énoncé.

1. `./lab.sh chaos cpu 300` (5 minutes de CPU à fond).
2. Suivez dans Prometheus **Alerts** puis dans Alertmanager, puis dans Teams (ou l'Inbox, canal `teams`). Chronométrez.
3. Ouvrez la carte : que contient-elle ? Qu'est-ce qui manque pour qu'elle soit vraiment utile ?

Corrigé. Pending vers 2 min (le `rate[2m]` doit dépasser 80 %, ce qui prend presque toute la fenêtre), firing 2 min plus tard, carte Teams 10 s après. La carte par défaut (`msteamsv2.default.text`) contient tous les labels et annotations : lisible mais verbeux. Il manque un lien vers le dashboard et le runbook en clair.

Partie 4 — Personnaliser le message 15 min

Énoncé.

1. Ouvrez `alertmanager/templates/formation.tpl` : deux blocs, `formation.title` et `formation.text`.
2. Utilisez-les dans le receiver `astreinte-teams` (`title:` et `text:`).
3. Ajoutez au bloc `formation.text` un lien vers le dashboard TP 4 :
`http://localhost:3000/d/formation-serveur-linux?var-instance={{ .Labels.instance }}`.
4. Rechargez, relancez un chaos CPU (ou attendez le `repeat_interval`), comparez.

Corrigé.

```
- name: astreinte-teams
  msteamsv2_configs:
    - webhook_url: http://inbox:8080/teams/astreinte
      send_resolved: true
      title: '{{ template "formation.title" . }}'
      text: '{{ template "formation.text" . }}'
```

Et dans le template :

```
{{ define "formation.text" }}
{{ range .Alerts -}}
{{ .Annotations.summary }}
Sévérité : {{ .Labels.severity }} | Instance : {{ .Labels.instance }}
Dashboard : http://localhost:3000/d/formation-serveur-linux?var-instance={{ .Labels.instance }}
{{ if .Annotations.runbook_url }}Runbook : {{ .Annotations.runbook_url }}{{ end }}
{{ end }}
{{ end }}
```

Astuce pour tester un template sans attendre une alerte : `amtool template render --template.glob='/etc/alertmanager/templates/*.tpl' --template.text='{{ template "formation.title" . }}'` (avec des données par défaut). Et pour tester un receiver sans alerte réelle : envoyer une alerte de test à Alertmanager directement :

```
curl -X POST http://localhost:9093/api/v2/alerts -H 'Content-Type: application/json' -d '[{
  "labels": {"alertname": "TestTeams", "severity": "critical", "instance": "test"},
  "annotations": {"summary": "Ceci est un test"}}]
```

Ce que je vérifie. Chacun a une carte Teams (ou Inbox `teams`) qui contient sa `summary` et le lien dashboard.

Module 14 — Alerting Grafana 25 min

Objectif. Comprendre l'alerting unifié de Grafana, ses différences avec Alertmanager, et quand choisir l'un ou l'autre.

Ce que je dis

Ce que c'est. Depuis Grafana 9, l'alerting « unifié » : des règles évaluées par Grafana lui-même, sur n'importe quelle source de données (Prometheus, mais aussi Loki, SQL, Elastic...), avec un Alertmanager embarqué (le même code) pour le routage. On retrouve les mêmes concepts sous d'autres noms :

Alertmanager	Grafana	
receiver	Contact point	où envoyer
route	Notification policy	qui reçoit quoi
silence	Silence	même chose
time_interval	Time interval (mute timing)	plages horaires
template	Notification template	même langage Go
règle Prometheus (<code>for</code>)	Alert rule (pending period, keep firing for)	
—	Recording rule Grafana	pré-calcul, écrit via remote write

Une règle Grafana. Une ou plusieurs requêtes (A, B...), puis des *expressions* : *Reduce* (une série → une valeur : last, mean, max), *Math* (`$A / $B * 100`), *Threshold* (`> 80`), *Classic condition*. La condition finale est une expression qui vaut 0 ou 1 par série. Multidimensionnel : une instance d'alerte par série, comme Prometheus. États : Normal, Pending, Alerting, NoData, Error (les deux derniers sont configurables : traiter NoData comme OK, Alerting ou NoData).

Deux modes dans l'interface : *Grafana-managed* (évalué par Grafana, ce qu'on vient de décrire) et *Data source-managed* (Grafana écrit la règle... dans Prometheus, via une API de rules ; nécessite Mimir/Cortex/Loki ou un Prometheus avec l'API d'écriture de règles). On reste sur Grafana-managed.

Quand utiliser quoi.

- Alertes sur métriques Prometheus, pures, versionnées dans Git, testées avec `promtool` : **Prometheus + Alertmanager**. Plus robuste (une alerte évaluée là où sont les données), pas de dépendance à Grafana, Alertmanager en HA trivial.
- Alertes multi-sources (un log Loki + une métrique), alertes SQL, équipes qui vivent dans Grafana et veulent cliquer, lien direct règle ↔ panel : **Grafana alerting**.
- Les deux coexistent très bien : Grafana peut aussi afficher les alertes Prometheus (source de données avec *Manage alerts via Alerting UI*, activé chez nous) et même router les alertes Prometheus vers son Alertmanager embarqué, ou un Alertmanager externe.
- Ce qu'il ne faut pas faire : dupliquer la même alerte des deux côtés.

Provisionner. Tout l'alerting Grafana se provisionne par fichier YAML dans `provisioning/alerting/`, par API (`/api/v1/provisioning/...`) ou Terraform. Chaque objet a un bouton *Export* dans l'interface qui génère le YAML.

Ce que je montre

Alerting → Alert rules : les règles Prometheus sont déjà visibles (section *Data source managed, Prometheus*), avec leur état. **Alerting → Notification configuration** : contact points, policies, templates, time intervals.

The screenshot shows the 'New alert rule' interface in Grafana. The left sidebar contains navigation links like Bookmarks, Starred, Dashboards, Explore, Drilldown, Alerting, Alert rules, Notification configuration, Silences, Active notifications, Settings, Connections, and Administration. The main content area is titled 'New alert rule' and contains three steps: 1. Enter alert rule name, 2. Define query and alert condition, and 3. Add folder and labels. Step 1 has a 'Name' input field. Step 2 includes a 'Prometheus' data source selector, a '10m' time range, and a 'Metric' selector. It also has a 'Label filters' section with 'Select label', 'IS ABOVE', and '0' options. Step 3 has a 'Folder' selector and a 'Labels' section.

Le formulaire de création d'une règle d'alerte Grafana 13

TP 8 — Alerting Grafana, de l'interface au code 45 min

Objectif. Créer une règle multi-dimensionnelle dans l'interface, un contact point Teams, une politique de notification, la tester, puis tout exporter et provisionner par fichier.

Partie 1 — Contact point et politique 10 min

Énoncé.

1. **Alerting** → **Notification configuration** → **Contact points** → **New contact point**. Nom `inbox-grafana`, intégration `Webhook`, URL `http://inbox:8080/webhook/grafana`. *Test* : le message arrive dans l'Inbox ? Sauvegardez.
2. Second contact point `teams-astreinte`, intégration `Microsoft Teams`, URL : votre URL Workflows ou `http://inbox:8080/teams/grafana`. *Test*, sauvegardez.
3. **Notification policies** : éditez la *Default policy* pour envoyer vers `inbox-grafana`. Puis *New notification policy* (route enfant) : matcher `severity = critical` → `teams-astreinte`, *Override grouping* non, *Override general timings* : group wait 10 s.

Corrigé. Le bouton *Test* envoie une alerte fictive `TestAlert` : première chose à faire pour tout nouveau contact point. Le webhook Grafana envoie le même format que le webhook Alertmanager, plus quelques champs (`orgId`, `title`, `message`, `state`) : on compare dans l'Inbox.

Partie 2 — Une règle dans l'interface 15 min

Énoncé. Alerting → Alert rules → New alert rule :

1. Nom : Boutique - taux d'erreur élevé (Grafana) .
2. Requête A (Prometheus, mode *Code*, type *Instant*) : le taux d'erreur 5xx **par instance**, en pourcentage.
3. Condition : *WHEN QUERY A IS ABOVE 5* (ou, en *Advanced options*, une expression *Reduce Last* puis *Threshold > 5*). *Preview alert rule condition*.
4. Dossier *Formation*, labels `severity=critical` , `team=boutique` , `source=grafana` .
5. Groupe d'évaluation `boutique-grafana` , intervalle 1 min, *Pending period* 2 min, *Keep firing for* 1 min. *Configure no data and error handling* : NoData → OK.
6. Notifications : contact point... rien à choisir si vous laissez la politique de notification décider (mode *Use notification policy* dans les *Advanced options*), sinon sélectionnez `teams-astreinte` .
7. Message : *Summary* `{{ $labels.instance }}` : `{{ printf "%.1f" $values.B.Value }}` % d'erreurs , *Description*, *Runbook URL*, et *Link dashboard and panel* vers le panneau *Taux d'erreur* du TP 5.
8. Sauvegardez. `./lab.sh chaos errors on` , attendez. Suivez la règle (état, historique), puis l'*Inbox*.

Corrigé. Le YAML équivalent est dans `solutions/jour-3/grafana-alerting.yml` . Points à commenter :

- *Instant* pour la requête : une alerte a besoin d'une valeur par série, pas d'une courbe. En mode *Range*, il faut obligatoirement un *Reduce*.
- `$values.B.Value` dans les annotations : `B` est le nom de l'expression *Reduce* (en mode simple, Grafana crée `B` et `C` tout seul). `$labels` fonctionne comme dans Prometheus.
- Le lien *dashboard/panel* ajoute les annotations `__dashboardUid__` et `__panelId__` : le message de notification contient un lien direct vers le graphique, et le panel affiche les alertes qui lui sont liées.
- La règle apparaît en *Alerting* avec deux instances (`shop-api-1`, `shop-api-2`). L'onglet *History* montre les transitions.

Partie 3 — La règle CPU et une mise en sourdine 5 min

Énoncé. Créez *Serveur - CPU élevé* (Grafana) (*warning*, *team infra*) sur la même logique avec un seuil à 80. Puis **Time intervals → Add time interval** *nuît* (22h-7h, Europe/Paris) et appliquez-le en *Mute timings* sur la politique par défaut. Que se passe-t-il pour l'intervalle 22:00 → 07:00 ?

Corrigé. Même contrainte qu'Alertmanager : deux intervalles, `22:00-24:00` et `00:00-07:00` .

Partie 4 — Tout en code 15 min

Énoncé.

1. Sur chaque objet (règle, contact point, politique, time interval) : *More* → *Export* (ou le bouton *Export* de la liste) au format YAML. Assemblez un fichier `grafana/provisioning/alerting/formation.yml` avec les sections `contactPoints` , `policies` , `groups` , `muteTimes` .
2. Supprimez les objets créés à la main dans l'interface. `docker compose restart grafana` .
3. Vérifiez que tout est revenu, en lecture seule (cadenas *Provisioned*).
4. Bonus : `curl -s -u admin:formation http://localhost:3000/api/v1/provisioning/alert-rules | jq .`

Corrigé. Fichier : `solutions/jour-3/grafana-alerting.yml` . Erreurs classiques : un `uid` manquant sur un contact point (obligatoire pour que les polices le référencent), un `folder` qui n'existe pas (Grafana le crée), `datasourceUid: prometheus` qui doit correspondre à l'uid provisionné. Et le fameux intervalle qui passe minuit : un fichier invalide **empêche Grafana de démarrer** (il redémarre en boucle, `docker compose logs grafana` montre `failure to parse file`). C'est plus brutal qu'Alertmanager, qui garde l'ancienne config.

Module 15 — Performances, limites, bonnes pratiques 35 min

Objectif. Savoir dimensionner, diagnostiquer un Prometheus qui souffre, et déployer proprement.

Ce que je dis

Ce qui coûte. Principalement : le nombre de séries actives (mémoire), le débit d'échantillons (CPU, disque), et les requêtes (CPU, mémoire au moment de la requête). Ordres de grandeur en 2026 sur une machine correcte : un Prometheus seul tient confortablement 1 à 2 millions de séries actives et quelques centaines de milliers d'échantillons par seconde. Mémoire : compter grossièrement 2 à 4 Ko par série active dans le head, plus le cache. Disque : 1 à 2 octets par échantillon (1 million de séries × 1 échantillon/15 s × 1,5 octet ≈ 8,5 Go par jour).

La TSDB. Le *head* en mémoire reçoit tout, adossé au **WAL** (journal sur disque, rejoué au démarrage : c'est pour ça qu'un gros Prometheus met plusieurs minutes à redémarrer). Toutes les deux heures, le head est écrit en **bloc** immuable (`01M3E7...` : chunks, index, meta.json, tombstones). Les blocs sont ensuite **compactés** en blocs plus gros (jusqu'à 10 % de la rétention). La rétention (`--storage.tsdb.retention.time` et/ou `.size`) supprime les blocs entiers. Supprimer des séries ciblées : l'API admin `delete_series` + `clean_tombstones` , et Prometheus 3.14 a ajouté une page dans l'interface pour ça.

Les séries périmées (staleness). Quand une série disparaît d'un scrape, Prometheus écrit un marqueur et la série disparaît des requêtes instantanées après 5 min. Une série qui change de labels toutes les minutes (un pod qui redémarre, un label avec un timestamp) crée une nouvelle série à chaque fois : c'est le *churn*, second tueur après la cardinalité.

Diagnostiquer.

- **Status → TSDB status** : nombre de séries, top des métriques et des labels par cardinalité.
- `prometheus_tsdb_head_series` , `rate(prometheus_tsdb_head_samples_appended_total[5m])` , `process_resident_memory_bytes{job="prometheus"}` , `prometheus_tsdb_compaction_duration_seconds` , `prometheus_engine_query_duration_seconds{slice="inner_eval"}` , `scrape_duration_seconds` , `scrape_samples_scraped` , `up` .
- `promtool tsdb analyze /prometheus` sur un bloc : les labels et métriques les plus coûteux.
- Un dashboard « Prometheus lui-même » : le 3662 communautaire, ou celui de kube-prometheus.

Limiter la casse.

- `sample_limit` , `label_limit` , `label_name_length_limit` , `label_value_length_limit` par job : Prometheus refuse un scrape qui dépasse (et `up` passe à 0, ce qui alerte).

- `metric_relabel_configs` avec `drop` pour ce qu'on ne veut pas.
- `--query.max-samples`, `--query.timeout`, `--query.max-concurrency`.
- Recording rules pour tout ce qui est affiché en boucle.
- Fenêtres de `rate` raisonnables, `$_rate_interval` dans Grafana, pas de dashboard auto-refresh à 5 s sur 24 h.

Déployer proprement.

- **Sécurité** : Prometheus n'a ni authentification ni chiffrement par défaut. Depuis 2.24, le `--web.config.file` ajoute TLS et basic auth (mots de passe bcrypt). En pratique on met un reverse proxy (nginx, Traefik, oauth2-proxy) devant, ou on ne l'expose pas et on passe par Grafana. Idem pour Alertmanager, les exporters et la Pushgateway. Les secrets dans des fichiers (`*_file`), jamais dans le YAML.
- **Haute disponibilité** : deux Prometheus identiques qui scrapent les mêmes cibles (avec `external_labels: replica: a|b`), deux ou trois Alertmanager en cluster ; Alertmanager déduplique. Grafana avec une base PostgreSQL et plusieurs répliques derrière un load balancer.
- **Mode agent** (`--agent`) : un Prometheus sans stockage ni requêtes, qui scrape et pousse en `remote_write`. Pour les sites distants et l'edge. Grafana Alloy fait la même chose (et les logs, et les traces, et OpenTelemetry).
- **Grafana** : PostgreSQL au lieu de SQLite dès qu'il y a plus de deux utilisateurs actifs, tout provisionné, un dossier par équipe, `admin` désactivé au profit du SSO, sauvegarde de la base.
- **Conventions** : nommage des métriques, labels `env / team / service` partout (via `external_labels` ou relabeling), un runbook par alerte, des tests de règles en CI.

Exercices 3.1 à 3.5 — Diagnostic 15 min

3.1 — Combien de séries actives ? Quelle métrique en a le plus ? Quel label a le plus de valeurs distinctes ? (TSDB status et requêtes) `prometheus_tsdb_head_series` (~4 000 chez nous) ;

`http_request_duration_seconds_bucket ; label le ou route selon le moment. topk(10, count by (__name__)) ({__name__=~".+"})`.

3.2 — Combien d'échantillons par seconde entrent ? Combien de mémoire consomme Prometheus ?

`rate(prometheus_tsdb_head_samples_appended_total[5m])` (~300/s) ;

`process_resident_memory_bytes{job="prometheus"}` (~150-300 Mo).

3.3 — Quel job coûte le plus cher au scrape ? Quel est le plus lent ? `sort_desc(sum by (job)`

`(scrape_samples_scraped))` → `redis` ou `shop-api` ; `sort_desc(max by (job)`

`(scrape_duration_seconds))` → `blackbox-http` (la sonde https).

3.4 — Mettez un `sample_limit: 100` sur le job `redis`. Rechargez. Que devient `up{job="redis"}` ?

Retirez-le. `up` passe à 0, et `scrape_samples_post_metric_relabeling` > 100 explique pourquoi.

Prometheus préfère perdre un job que se noyer.

3.5 — Quelle requête de ce matin était la plus lente ? Comparez la version brute et la recording rule

avec `?stats=all : curl -s 'http://localhost:9090/api/v1/query?stats=all' --data-urlencode 'query=histogram_quantile(0.95, sum by (route, le)`

`(rate(http_request_duration_seconds_bucket[5m]))'` | `jq .data.stats totalQueryableSamples` et `timings.execTotalTime` : la brute lit des milliers d'échantillons, la recording rule quelques dizaines.

TP 9 — Sauvegarde, restauration, longue durée, sécurité 45 min

Objectif. Sauvegarder et restaurer Prometheus et Grafana, envoyer les métriques importantes vers un stockage longue durée, protéger Prometheus par mot de passe.

Mise en situation. Un audit demande : « si le serveur de monitoring brûle, en combien de temps le remettez-vous ? Et avez-vous 13 mois d'historique ? Et qui peut lire vos métriques ? »

Partie 1 — Snapshot et restauration de Prometheus 15 min

Énoncé.

1. `./lab.sh snapshot` (appelle `POST /api/v1/admin/tsdb/snapshot`, possible grâce à `--web.enable-admin-api`). Listez les snapshots : `docker compose exec prometheus ls /prometheus/snapshots/`. Que contient un snapshot ?
2. Notez le nombre de séries de `shop_orders_total` d'il y a 5 min : `count(shop_orders_total offset 5m)`.
3. Simulez la catastrophe et restaurez :

```
SNAP=<nom du snapshot>
docker compose stop prometheus
docker compose run --rm --no-deps --user root --entrypoint sh prometheus -c \
  "cd /prometheus && mv snapshots /tmp/ && rm -rf .* && cp -a /tmp/snapshots/$SNAP/. . && mv /tmp/snapshots . && chown -R nobody:nobody /prometheus"
docker compose start prometheus
```

4. Vérifiez que l'historique est là (`count(shop_orders_total offset 5m)` renvoie toujours des séries). Qu'a-t-on perdu ? (Le `chown nobody` correspond à l'utilisateur de l'image officielle.)

Corrigé. Un snapshot est un dossier avec des liens durs vers les blocs existants plus un bloc pour le head : quasi instantané, et il ne coûte que le head en espace. On perd ce qui a été ingéré entre le snapshot et la restauration. Une stratégie réelle : snapshot + `rsync / restic` du dossier `snapshots/` vers un stockage objet, toutes les heures ou toutes les nuits, puis suppression du snapshot local. Alternative : sauvegarder `data/` directement à froid (Prometheus arrêté) ou avec un snapshot du volume (LVM, EBS). Ne jamais copier `data/` à chaud sans snapshot : le WAL bouge.

Partie 2 — Sauvegarde de Grafana 10 min

Énoncé.

1. Où est la base de Grafana ? Copiez-la : `docker compose cp grafana:/var/lib/grafana/grafana.db ./grafana-backup.db`. Quelle taille ? Qu'y a-t-il dedans ?
2. Exportez tous les dashboards via l'API en un script (bash ou PowerShell) : listez avec `/api/search?type=dash-db`, puis `/api/dashboards/uid/<uid>` pour chacun, en écrivant un fichier JSON par dashboard.
3. Discussion : qu'est-ce qui n'est **pas** dans les dashboards JSON et qu'il faut aussi sauvegarder ?

Corrigé.

```
mkdir -p backup && for uid in $(curl -s -u admin:formation 'http://localhost:3000/api/search?
type=dash-db' | jq -r '.[].uid'); do
  curl -s -u admin:formation "http://localhost:3000/api/dashboards/uid/$uid" | jq .dashboard >
  "backup/$uid.json"
done
```

Ce qui n'est pas dans les JSON : utilisateurs, teams, permissions, sources de données (et leurs secrets, chiffrés dans la base avec `secret_key` de `grafana.ini`), alerting, préférences, plugins. D'où : soit on sauvegarde la base (SQLite : copie à chaud acceptable avec `sqlite3 .backup`, mieux avec Grafana arrêté ; PostgreSQL : `pg_dump`), soit, et c'est la bonne réponse, **tout est provisionné et la sauvegarde, c'est Git**. La base ne contient alors que de l'état reconstituable.

Partie 3 — Remote write vers un Prometheus longue durée 10 min

Énoncé.

1. `./lab.sh longterm` démarre un second Prometheus (port 9095) avec 90 jours de rétention et le récepteur remote write activé.
2. Ajoutez à `prometheus.yml` un bloc `remote_write` vers `http://prometheus-longterm:9090/api/v1/write` qui n'envoie **que** les métriques métier (`shop_*`), les recording rules (`job:*`, `instance:*`, `route:*`) et `up`. Validez, rechargez.
3. Après une minute, sur `http://localhost:9095` : quelles métriques sont présentes ? Combien de séries ? Comparez avec le Prometheus principal.
4. `prometheus_remote_storage_samples_total`, `prometheus_remote_storage_samples_failed_total`, `prometheus_remote_storage_highest_timestamp_in_seconds` - `prometheus_remote_storage_queue_highest_sent_timestamp_seconds` (le retard).

Corrigé. `solutions/jour-3/prometheus-remote-write.yml`. Une quinzaine de noms de métriques côté longue durée (`count(count by (__name__) ({__name__=~".+"}))`) contre plus de 800 côté principal. C'est le pattern : du brut local avec une rétention courte, de l'agrégé envoyé au loin avec une rétention longue. Le récepteur remote write d'un Prometheus est pratique pour un lab ou un petit site ; en production, la cible est Mimir, Thanos Receive, VictoriaMetrics ou un service managé (module 16). Remote Write 2.0 (Prometheus 3) réduit nettement la bande passante (chaînes internées, compression) et transporte métadonnées et native histograms.

Partie 4 — Un mot de passe sur Prometheus (bonus, 10 min)

Énoncé.

1. Créez `prometheus/web.yml` avec `basic_auth_users` : utilisateur `admin`, mot de passe `formation` (hash bcrypt fourni dans le guide, ou généré avec `htpasswd -nBC 10 ""`).
2. Ajoutez `--web.config.file=/etc/prometheus/web.yml` au service `prometheus` du compose, `docker compose up -d prometheus`. Vérifiez : `http://localhost:9090` demande un mot de passe ; `promtool check web-config` valide le fichier.
3. Qu'est-ce qui casse ? Réparez-le.

Corrigé. `solutions/jour-3/web.yml`. Ce qui casse : Grafana (source de données → *Basic auth* à activer avec les identifiants, ou dans le provisioning `basicAuth: true`, `basicAuthUser`, `secureJsonData.basicAuthPassword`), `./lab.sh reload` (ajouter `-u admin:formation`), le Prometheus

longue durée si on lui avait mis un mot de passe aussi (`basic_auth` dans `remote_write`), et Alertmanager ne parle pas à Prometheus, donc rien de ce côté. Leçon : la sécurité se fait au début, pas à la fin. On retire le `web.config.file` avant le war game pour ne pas se compliquer la vie.

Module 16 — Mise à l'échelle et écosystème 20 min

Objectif. Savoir quoi faire quand un Prometheus ne suffit plus, et où va l'écosystème.

Ce que je dis

Les signaux. OOM récurrents ou compaction qui n'en finit pas → trop de séries : d'abord réduire (cardinalité, `drop`, agrégation). Besoin de plus de 30-60 jours → stockage longue durée. Plusieurs sites/clusters et une vue globale → fédération ou stockage central. Besoin de HA « vraie » (pas de trou pendant un redémarrage) → duo de Prometheus, ou stockage distribué.

Les options, dans l'ordre.

1. **Sharding fonctionnel** : un Prometheus par équipe, par cluster, par type (infra / applis). Simple, robuste, mais des vues séparées.
2. **Fédération** : un Prometheus central qui scrape `/federate` de Prometheus régionaux (avec un `match[]` sur des recording rules agrégées). Ancien, limité, mais suffisant pour une vue globale légère.
3. **Remote write vers un stockage distribué** :
 - **Grafana Mimir** (issu de Cortex) : horizontalement scalable, multi-tenant, stockage objet, c'est le moteur de Grafana Cloud.
 - **Thanos** : sidecar à côté de chaque Prometheus, blocs envoyés en stockage objet, un *Querier* global, déduplication, downsampling. Très répandu, plus de composants.
 - **VictoriaMetrics** : simple à opérer, très économe, PromQL étendu (MetricsQL). Version single-node ou cluster.

Les trois exposent une API compatible Prometheus : Grafana ne voit pas la différence.

4. **Managé** : Grafana Cloud, Amazon Managed Prometheus, Google Managed Prometheus, Azure Monitor managed Prometheus.

OpenTelemetry. Le standard d'instrumentation pour les trois signaux. Prometheus 3 reçoit nativement de l'OTLP (`--web.enable-otlp-receiver`) et sait gérer les noms de métriques avec points et UTF-8 (nouvelle syntaxe `{"service.name"="x"}`). Le Collector OTel (ou Grafana Alloy) peut scraper des cibles Prometheus, recevoir de l'OTLP, et envoyer en remote write. En 2026, instrumenter en OpenTelemetry et stocker dans Prometheus/Mimir est la combinaison la plus courante pour du neuf.

Loki et Tempo. Les mêmes idées appliquées aux logs (Loki indexe les labels, pas le texte) et aux traces (Tempo). Grafana les corrèle : d'un pic de latence (Prometheus) vers les traces (exemplars) vers les logs, en quelques clics. C'est la suite logique de cette formation.

Kubernetes. kube-prometheus-stack : Prometheus Operator, `ServiceMonitor`, `PodMonitor`, `PrometheusRule`, kube-state-metrics, node-exporter, Grafana avec des dizaines de dashboards, le tout en un `helm install`. Tout ce qu'on a vu s'applique ; la découverte de services fait le reste.

Ce que je montre

Le schéma du deck : Prometheus locaux → remote write → Mimir/Thanos/VictoriaMetrics → Grafana.
Et les liens de l'annexe C.

17h05 — War game et évaluation finale 25 min

Le war game (15 min). Les stagiaires ne touchent plus à la configuration. Je casse la boutique d'une façon qu'ils ne connaissent pas, en ciblant **une seule instance** (le script `lab.sh` casse les deux instances ; moi, je vise) :

```
curl -X POST http://localhost:5002/chaos/latency/on          # shop-api-2 seulement
curl -X POST "http://localhost:5001/chaos/cpu?seconds=240"  # CPU depuis shop-api-1
docker compose stop redis-exporter                         # un exporter qui disparaît
```

Chaque stagiaire (ou binôme) doit, en moins de cinq minutes et par écrit : (1) dire ce qui est cassé, (2) sur quelle instance, (3) depuis quand, (4) quelle alerte a sonné et quelle alerte aurait dû sonner, (5) quelle serait la première action. Ils ont Grafana, Prometheus, Alertmanager, l'Inbox. Puis on débriefe : qui a trouvé quoi, avec quel outil, et surtout ce qui manquait (souvent : une alerte sur `redis_up`, un panneau par instance, un lien dashboard → logs).

Variante si le groupe est fort : ils écrivent l'alerte manquante avant de partir.

L'évaluation (10 min). Le questionnaire de fin de formation reprend les questions du positionnement du jour 1 plus une dizaine de questions techniques (annexe A). Correction à l'oral, tout de suite.

Pour finir. Ce qu'ils emportent : le dépôt (branche `formation-2026`), leur stack, les trois guides, les liens de l'annexe C. Ce que je leur conseille de faire lundi : choisir **un** service chez eux, l'instrumenter en RED, un dashboard, deux alertes symptômes, un runbook. Pas plus. Le reste viendra.

Je rappelle le questionnaire de satisfaction Sparks et je reste dix minutes pour les questions individuelles.

Annexes

Annexe A — Questionnaire d'évaluation (avec corrigé)

Le même questionnaire sert au positionnement du jour 1 (sans corrigé) et à l'évaluation finale. Une réponse par question, 15 questions, 20 minutes.

- Prometheus collecte les métriques en mode : (a) push (b) pull (c) les deux à parts égales. **b.** Le push existe via la Pushgateway ou le remote write, mais le modèle est le pull.
- Une série temporelle est identifiée par : (a) son nom (b) son nom et son timestamp (c) son nom et l'ensemble de ses labels. **c.**
- Sur un Counter, la fonction à utiliser pour obtenir un débit est : (a) `avg()` (b) `rate()` (c) `sum()`. **b.**
- `http_requests_total{status=~"5.."}{}` sélectionne : (a) les codes 5xx (b) les codes commençant par 5 (c) tout ce qui contient un 5. **a.** la regex est ancrée.
- Le fichier à valider avant de recharger Prometheus se vérifie avec : (a) `prometheus --check` (b) `promtool check config` (c) `docker compose config`. **b.**
- Pour un p95 sur un histogramme, `histogram_quantile` a besoin que l'agrégation conserve le label : (a) `le` (b) `quantile` (c) `instance`. **a.**
- Une alerte Prometheus se déclenche quand : (a) la valeur dépasse 1 (b) l'expression renvoie au moins une série (c) `for` est écoulé, même sans résultat. **b.** (puis `for`).
- `group_wait` dans Alertmanager sert à : (a) attendre avant la première notification d'un groupe (b) espacer les répétitions (c) définir un silence. **a.**
- En 2026, Alertmanager notifie Microsoft Teams via : (a) le connecteur Office 365 (b) `msteamsv2_configs` et un flux Workflows (c) un mail. **b.**
- Dans Grafana, une variable multi-valeur s'utilise dans une requête avec : (a) `=` (b) `=~` (c) `!=`. **b.**
- Pour fusionner deux requêtes en une seule table Grafana, on utilise : (a) une jointure PromQL (b) la transformation *Merge* (c) deux panels. **b.**
- Un dashboard provisionné par fichier : (a) ne peut jamais être édité (b) est rechargé depuis le fichier, l'édition UI est optionnelle (c) est stocké uniquement en mémoire. **b.**
- Le principal facteur de consommation mémoire de Prometheus est : (a) le nombre de cibles (b) le nombre de séries actives (c) la rétention. **b.**
- Pour un historique de 13 mois multi-clusters, on choisit : (a) augmenter la rétention à 400 j (b) remote write vers Mimir/Thanos/VictoriaMetrics (c) un Prometheus par mois. **b.**
- Un `up == 0` sur un job blackbox signifie : (a) la cible sondée est down (b) l'exporter blackbox ne répond pas (c) la sonde a échoué. **b.** ; la sonde, c'est `probe_success`.

Questions ouvertes (à l'oral) : « décrivez l'architecture », « citez trois labels interdits », « qu'est-ce qu'une alerte symptôme ».

Annexe B — Aide-mémoire

Commandes du lab

```
./lab.sh up | down | reset | status | reload | check | test | logs <svc>
./lab.sh chaos latency|errors|leak on|off      ./lab.sh chaos cpu 300      ./lab.sh chaos reset
./lab.sh traffic 20                          ./lab.sh batch      ./lab.sh longterm      ./lab.sh snapshot
docker compose exec prometheus promtool check config /etc/prometheus/prometheus.yml
docker compose exec prometheus promtool check rules /etc/prometheus/rules/*.yaml
docker compose exec prometheus sh -c 'cd /etc/prometheus/tests && promtool test rules *.yaml'
docker compose exec alertmanager amtool check-config /etc/alertmanager/alertmanager.yml
docker compose exec alertmanager amtool config routes show --
config.file=/etc/alertmanager/alertmanager.yml
docker compose exec alertmanager amtool config routes test --
config.file=/etc/alertmanager/alertmanager.yml \
    severity=critical
docker compose exec alertmanager amtool alert      # l'URL vient de /etc/amtool/config.yml
docker compose exec alertmanager amtool silence add alertname=X -d 1h -c "commentaire"
curl -X POST localhost:9090/-/reload      curl -X POST localhost:9093/-/reload
curl -X POST localhost:9090/api/v1/admin/tsdb/snapshot
```

PromQL

Besoin	Requête
Cibles down	<code>up == 0</code>
Débit	<code>sum by (route) (rate(http_requests_total[5m]))</code>
Taux d'erreur	<code>sum(rate(x{status=~"5.."}[5m])) / sum(rate(x[5m]))</code>
p95	<code>histogram_quantile(0.95, sum by (le) (rate(x_bucket[5m])))</code>
Latence moyenne	<code>rate(x_sum[5m]) / rate(x_count[5m])</code>
Incréments sur 1 h	<code>increase(x[1h])</code>
CPU %	<code>100 * (1 - avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])))</code>
Mémoire %	<code>100 * (1 - node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)</code>
Disque %	<code>100 * (1 - node_filesystem_avail_bytes / node_filesystem_size_bytes)</code>
Disque plein dans 24 h	<code>predict_linear(node_filesystem_avail_bytes[6h], 86400) < 0</code>
Pic sur 1 h	<code>max_over_time(x[1h])</code>
Métrique absente	<code>absent(up{job="x"})</code>
Jointure info	<code>x * on (instance) group_left(version) app_info</code>
Top cardinalité	<code>topk(10, count by (__name__) ({__name__=~".+"}))</code>
Valeurs distinctes	<code>count(count by (label) (x))</code>
Comparer à hier	<code>rate(x[5m]) offset 1d</code>

Variables Grafana

`$__rate_interval` (fenêtre pour rate), `$__interval`, `$__range`, `$__from`, `$__to`, `$variable` / `${variable}`, `label_values(metric, label)`, `=~"$var"` en multi-valeur.

Alertmanager : durées par défaut

`group_wait` 30 s, `group_interval` 5 m, `repeat_interval` 4 h, `resolve_timeout` 5 m.

Ports

Prometheus 9090 · Alertmanager 9093 · Grafana 3000 · Pushgateway 9091 · Node Exporter 9100 · Blackbox 9115 · Redis exporter 9121 · cAdvisor 8080 (8085 dans le lab) · Inbox 8080 (lab) · shop-api 5001/5002 (lab)

Annexe C — Ressources

- Documentation Prometheus : <https://prometheus.io/docs/> (en particulier *Querying* → *Functions*, *Configuration*, *Alerting* → *Configuration*, *Migration 2.x* → *3.x*)
- Documentation Grafana : <https://grafana.com/docs/grafana/latest/> (*Dashboards* → *Build dashboards*, *Alerting*, *Administration* → *Provisioning*)
- Alertmanager : <https://prometheus.io/docs/alerting/latest/configuration/>
- Catalogue d'exporters : <https://prometheus.io/docs/instrumenting/exporters/>
- Conventions de nommage : <https://prometheus.io/docs/practices/naming/>
- Bonnes pratiques d'alerting : <https://prometheus.io/docs/practices/alerting/>
- Règles d'alerte prêtes à l'emploi : <https://samber.github.io/awesome-prometheus-alerts/>
- Dashboards communautaires : <https://grafana.com/grafana/dashboards/> (Node Exporter Full : 1860)
- Teams via Workflows (Grafana) : <https://grafana.com/docs/grafana/latest/alerting/configure-notifications/manage-contact-points/integrations/configure-teams/>
- Google SRE Book, chapitre *Monitoring Distributed Systems* : <https://sre.google/sre-book/monitoring-distributed-systems/>
- kube-prometheus-stack : <https://github.com/prometheus-community/helm-charts>
- Mimir, Thanos, VictoriaMetrics : grafana.com/oss/mimir, thanos.io, victoriametrics.com
- Le blog de Stéphane Robert (observabilité, en français) : <https://blog.stephane-robert.info/docs/outils/observabilite/>
- Le dépôt de la formation : <https://github.com/yparent/formation-observabilite-lab> (branche `formation-2026`)

Annexe D — Dépannage pendant la formation

Symptôme	Cause probable	Remède
<code>./lab.sh up</code> échoue sur un port	Port déjà utilisé	<code>lsof -i :3000</code> / <code>netstat -ano</code> ; arrêter ou changer le port gauche dans compose

Symptôme	Cause probable	Remède
Grafana « database is locked » ou lent	SQLite sur un volume partagé lent (Windows)	Patienter ; ou <code>./lab.sh reset</code>
Cible <code>DOWN</code> avec <code>connection refused</code>	Conteneur pas démarré ou mauvais port	<code>./lab.sh status</code> , <code>./lab.sh logs <svc></code>
Reload refusé	YAML invalide	<code>./lab.sh check</code> , lire le message (ligne, colonne)
Reload sans effet	Oubli du reload, ou fichier édité au mauvais endroit	vérifier Status → Configuration
<code>promtool</code> introuvable	Lancé sur la machine au lieu du conteneur	<code>docker compose exec prometheus promtool ...</code>
Chaîne <code>\$</code> mangée dans compose	Interpolation	doubler : <code>\$\$</code>
Panel Grafana vide sans « No data »	Option invalide (ex. <code>legend.displayMode: hidden</code>)	passer par l'UI, ou <code>showLegend: false</code>
<code>histogram_quantile</code> renvoie NaN	<code>le</code> perdu dans l'agrégation	<code>sum by (le)</code>
Variable Grafana sans valeur	mauvaise requête <code>label_values</code> ou <code>=</code> au lieu de <code>=~</code>	tester dans Explore
Alerte jamais <code>firing</code>	<code>for</code> trop long, ou expression vide (division par vide)	tester l'expression dans Prometheus, <code>or vector(0)</code>
Notification Teams absente	flux Workflows en erreur	Power Automate → historique d'exécution
Grafana redémarre en boucle	fichier de provisioning invalide	<code>docker compose logs grafana</code> , corriger, <code>restart</code>
Codespace lent	machine 2 cœurs	<code>./lab.sh traffic 3</code> , éviter cAdvisor
Node Exporter : chiffres bizarres	Docker Desktop (VM)	expliquer, c'est normal
Build de l'app impossible (pas de réseau)	proxy d'entreprise	pré-construire l'image la veille, <code>docker save</code> / <code>docker load</code>

Annexe E — Créer le flux Teams Workflows (pas à pas)

1. Dans Teams, ouvrir le canal cible, cliquer sur ... à côté de son nom → **Workflows**.
2. Chercher « **webhook** » et choisir « **Post to a channel when a webhook request is received** » (en français : « Publier dans un canal lorsqu'une demande de webhook est reçue »).
3. Nommer le flux (`Alertes Prometheus`), **Suivant**. Vérifier l'équipe et le canal, **Ajouter un flux de travail**.
4. Copier l'URL affichée (`https://prod-xx.westeurope.logic.azure.com:443/workflows/...`). Elle ne sera plus affichée : la stocker dans un gestionnaire de secrets.

5. Canal privé : ouvrir le flux dans Power Automate, étape « Send each adaptive card » → action « Post your own adaptive card as the Flow bot to a channel » → *Post as* : **User**. Enregistrer.
6. Tester : `curl -X POST '<URL>' -H 'Content-Type: application/json' -d @carte.json` avec une carte adaptative minimale :

```
{ "type": "message", "attachments":  
  [ { "contentType": "application/vnd.microsoft.card.adaptive", "content": {  
    "$schema": "http://adaptivecards.io/schemas/adaptive-  
card.json", "type": "AdaptiveCard", "version": "1.4",  
    "body": [ { "type": "TextBlock", "text": "Test depuis la formation", "weight": "Bolder" } ] } } ] }
```

7. Dans Alertmanager : `msteamsv2_configs: [{webhook_url_file: /etc/alertmanager/secrets/teams.url}]` . Dans Grafana : contact point *Microsoft Teams*, champ *URL*.

Limites : pas de mise en forme personnalisée de la carte depuis Grafana (titre et message seulement) ; les flux Workflows sont soumis aux limites Power Automate du tenant (nombre d'appels par jour selon la licence) ; un flux appartient à un utilisateur, prévoir un compte de service.